



Università degli Studi di Padova

Laurea: Informatica

Corso: Ingegneria del Software

Anno Accademico: 2025/26



Gruppo 17

Nome: BitByBit

Email: swe.bitbybit@gmail.com

Specifica Tecnica



Stato:	In lavorazione
Versione:	0.2.1
Responsabile:	Gabriele Scaggiante
Redattori:	Gabriele Scaggiante
	Dennis Parolin
	Ferdinando Fracasso
	Giovanni Visentin
	Marco Sanguin
Verificatori:	Riccardo Manisi
	Dennis Parolin
	Marco Sanguin
	Giovanni Visentin
Destinatari:	Miriade srl
	Prof. Tullio Vardanega
	Prof. Riccardo Cardin
	Gruppo BitByBit



Registro delle modifiche

Versione	Data	Autore	Descrizione	Verificatore
0.4.0	2026-04-17	Riccardo Manisi	Redatta la sezione Architettura InvioSOS	Gabriele Scaggiante
0.3.0	2026-04-17	Riccardo Manisi	Redatta la sezione Architettura auth	Ferdinando Fracasso
0.2.1	2026-04-12	Marco Sanguin	Redatta la sezione Architettura luoghi sicuri	Riccardo Manisi
0.2.0	2026-04-12	Marco Sanguin	Redatta la sezione Architettura funzionalità Dead man's switch	Ferdinando Fracasso
0.1.9	2026-04-12	Dennis Parolin	Creata la sezione Architettura Back End con relative sottosezioni Autenticazione , Luoghi sicuri e Materiale informativo	Giovanni Visentin
0.1.8	2026-04-11	Marco Sanguin	Aggiunta la sezione Stato dei requisiti funzionali	Ferdinando Fracasso
0.1.7	2026-04-10	Marco Sanguin	Redatta la sezione Architettura Area Informativa	Ferdinando Fracasso
0.1.6	2026-04-09	Ferdinando Fracasso	Redatta la sezione Design pattern utilizzati	Giovanni Visentin
0.1.5	2026-04-06	Ferdinando Fracasso	Redatta la sezione Architettura funzionalità diari	Marco Sanguin
0.1.4	2026-04-05	Giovanni Visentin	Aggiunta della funzionalità dei contatti fidati	Dennis Parolin



0.1.3	2026-04-04	Gabriele Scaggiante	Iniziata la sezione Architettura frontend , completata Architettura chatbot	Marco Sanguin
0.1.2	2026-04-02	Dennis Parolin	Redatta la sezione Architettura logica e Architettura di deployment	Marco Sanguin
0.1.1	2026-03-31	Gabriele Scaggiante	Aggiunta di EventBridge, SAM, CloudFormation e Docker nella sezione tecnologie.	Dennis Parolin
0.1.0	2026-03-30	Gabriele Scaggiante	Creazione del documento. Scrittura dell'introduzione e delle tecnologie utilizzate.	Riccardo Manisi



Indice

1	Introduzione	11
1.1	Scopo del documento	11
1.2	Riferimenti	11
1.2.1	Riferimenti normativi	12
1.2.2	Riferimenti informativi	12
2	Tecnologie	13
2.1	Framework	13
2.1.1	Flutter	13
2.2	Linguaggi di programmazione	13
2.2.1	Dart	13
2.2.2	Python	14
2.3	Servizi AWS	14
2.3.1	Lambda	14
2.3.2	API Gateway	14
2.3.3	DynamoDB	14
2.3.4	S3	15
2.3.5	Cognito	15
2.3.6	Bedrock	15
2.3.7	SES	15
2.3.8	CloudWatch	15
2.3.9	EventBridge	16
2.3.10	SAM	16
2.3.11	CloudFormation	16
2.4	Librerie	16
2.4.1	http.dart	16
2.4.2	image_picker.dart	17
2.4.3	path_provider.dart	17
2.5	Tecnologie di analisi statica	17
2.5.1	SonarQube	17
2.6	Tecnologie di analisi dinamica	17
2.6.1	Pytest	17
2.6.2	Moto	18
2.7	Altro	18
2.7.1	Ollama	18
2.7.2	Docker	18

3	Architettura	18
3.1	Architettura logica	18
3.1.1	Livello Client (Frontend Mobile)	19
3.1.2	Livello di Servizio (Backend)	19
3.2	Design pattern utilizzati	20
3.2.1	Model-view-viewmodel	20
3.2.2	Observer	20
3.2.3	Proxy	21
3.2.4	Singleton	21
3.3	Architettura di deployment	22
3.3.1	Client Mobile	22
3.3.2	Infrastruttura AWS (Serverless Cloud)	22
3.3.3	Orchestrazione Event-Driven	23
3.3.4	Infrastructure as Code (IaC)	23
3.4	Architettura Front End	23
3.4.1	Autenticazione	23
3.4.2	Invio SOS	31
3.4.3	Chatbot	35
3.4.4	Diario criptato e diario fittizio	45
3.4.5	Contatti Fidati	54
3.4.6	Luoghi Sicuri	59
3.4.7	Dead Man's Switch	63
3.4.8	Area Informativa	67
3.5	Architettura Back End	78
3.5.1	Autenticazione	78
3.5.2	Luoghi sicuri	80
3.5.3	Materiale informativo	81
3.5.4	Diario criptato e fittizio	83
3.5.5	Chatbot	85
3.5.6	Contatti fidati, invio SOS e Dead man's switch	88
4	Stato dei requisiti funzionali	90
4.1	Requisiti Funzionali obbligatori	90
4.2	Requisiti Funzionali opzionali	104



Elenco delle tabelle

2	Tabella dello stato dei requisiti funzionali obbligatori	103
3	Tabella dello stato dei requisiti funzionali opzionali	104

Elenco delle figure

1	Diagramma delle classi relativo all'autenticazione	24
2	Diagramma della classe <code>AuthScreen</code>	25
3	Diagramma della classe <code>AuthHeaderWidget</code>	25
4	Diagramma della classe <code>GoogleLoginButtonWidget</code>	26
5	Diagramma della classe <code>User</code>	27
6	Diagramma della classe <code>UserDTO</code>	28
7	Diagramma della classe <code>AuthViewModel</code>	29
8	Diagramma della classe <code>AuthRepository</code>	30
9	Diagramma della classe <code>AuthenticationService</code>	31
10	Diagramma delle classi relativo alla funzionalità dell'invio email di soccorso	32
11	Diagramma della classe <code>SosBottomSheetWidget</code>	32
12	Diagramma della classe <code>SosAlertViewModel</code>	33
13	Diagramma della classe <code>SosAlertRepository</code>	33
14	Diagramma della classe <code>SosAlertService</code>	33
15	Diagramma della classe <code>Command<T></code>	34
16	Diagramma della classe <code>Command0<T></code>	34
17	Diagramma della classe <code>Command1<T, A></code>	35
18	Diagramma delle classi relativo alle funzionalità del Chatbot	36
19	Diagramma della classe <code>ChatbotScreen</code>	36
20	Diagramma della classe <code>ChatWidget</code>	37
21	Diagramma della classe <code>ChatHistoryWidget</code>	37
22	Diagramma della classe <code>ChatbotModeToggleWidget</code>	38
23	Diagramma della classe <code>ChatbotSendMessageWidget</code>	38
24	Diagramma della classe <code>ChatbotCreateChatWidget</code>	39
25	Diagramma della classe <code>ChatbotViewModel</code>	39
26	Diagramma della classe <code>Chat</code>	40
27	Diagramma della classe <code>ProxyChat</code>	40
28	Diagramma della classe <code>LocalChat</code>	41
29	Diagramma della classe <code>ChatMessage</code>	41
30	Diagramma della classe <code>ChatMode</code>	42
31	Diagramma della classe <code>MessageType</code>	42
32	Diagramma della classe <code>MessageResponse</code>	43
33	Diagramma della classe <code>ChatbotRepository</code>	43



34	Diagramma della classe ChatbotService	44
35	Diagramma della classe ChatDTO	44
36	Diagramma delle classi relativo alle funzionalità del diario criptato	45
37	Diagramma della classe DiaryAccessScreen	46
38	Diagramma della classe PasswordFormWidget	46
39	Diagramma della classe DiaryAccessViewModel	47
40	Diagramma della classe DiaryAccountRepository	47
41	Diagramma della classe DiaryAccountService	47
42	Diagramma della classe DiaryAccessResult	48
43	Diagramma della classe DiaryScreen	48
44	Diagramma della classe NoteListWidget	48
45	Diagramma della classe NoteActionsWidget	49
46	Diagramma della classe DiaryViewModel	49
47	Diagramma della classe DiarySession	50
48	Diagramma della classe DiaryType	50
49	Diagramma della classe Note	51
50	Diagramma della classe ProxyNote	51
51	Diagramma della classe LocalNote	52
52	Diagramma della classe NoteElement	52
53	Diagramma della classe NoteTextElement	52
54	Diagramma della classe NoteImgElement	53
55	Diagramma della classe NoteAudioElement	53
56	Diagramma della classe NoteService	54
57	Diagramma della classe NoteRepository	54
58	Diagramma della classe NoteDTO	54
59	Diagramma delle classi relativo alle funzionalità dei Contatti Fidati	55
60	Diagramma della classe TrustedContactScreen	56
61	Diagramma della classe TrustedContactListWidget	56
62	Diagramma della classe TrustedContactFormWidget	56
63	Diagramma della classe TrustedContactActionsWidget	57
64	Diagramma della classe TrustedContactViewModel	57
65	Diagramma della classe TrustedContact	58
66	Diagramma della classe TrustedContactRepository	58
67	Diagramma della classe TrustedContactService	59
68	Diagramma della classe TrustedContactDTO	59
69	Diagramma delle classi complessivo della funzionalità Luoghi Sicuri	60
70	Diagramma della classe SafePlaceMapScreen	61
71	Diagramma della classe SafePlaceMapWidget	61
72	Diagramma della classe SafePlaceViewModel	61
73	Diagramma della classe SafePlace	62



74	Diagramma della classe SafePlaceRepository	62
75	Diagramma della classe SafePlaceService	63
76	Diagramma della classe SafePlaceDTO	63
77	Diagramma delle classi relativo alla funzionalità del Dead Man's Switch . .	64
78	Diagramma della classe DeadManScreen	65
79	Diagramma della classe DeadManFormWidget	65
80	Diagramma della classe DeadManViewModel	66
81	Diagramma della classe DeadManSettings	66
82	Diagramma della classe DeadManRepository	67
83	Diagramma della classe DeadManService	67
84	Diagramma della classe DeadManSettingsDTO	67
85	Diagramma delle classi complessivo per l'Area Informativa	68
86	Diagramma della classe InfoHubScreen	69
87	Diagramma della classe CommunityScreen	69
88	Diagramma della classe CommunityListWidget	69
89	Diagramma della classe CommunityViewModel	70
90	Diagramma della classe Community	70
91	Diagramma della classe CommunityRepository	70
92	Diagramma della classe CommunityService	71
93	Diagramma della classe CommunityDTO	71
94	Diagramma della classe LegislationScreen	71
95	Diagramma della classe LegislationListWidget	71
96	Diagramma della classe LegislationViewModel	72
97	Diagramma della classe Legislation	72
98	Diagramma della classe LegislationRepository	73
99	Diagramma della classe LegislationService	73
100	Diagramma della classe LegislationDTO	73
101	Diagramma della classe InfoMaterialScreen	73
102	Diagramma della classe InfoMaterialListWidget	74
103	Diagramma della classe InfoMaterialViewModel	74
104	Diagramma della classe InfoMaterial	74
105	Diagramma della classe InfoMaterialRepository	75
106	Diagramma della classe InfoMaterialService	75
107	Diagramma della classe InfoMaterialDTO	75
108	Diagramma della classe EmergencyContactScreen	76
109	Diagramma della classe EmergencyContactListWidget	76
110	Diagramma della classe EmergencyContactViewModel	76
111	Diagramma della classe EmergencyContact	77
112	Diagramma della classe EmergencyContactRepository	77
113	Diagramma della classe EmergencyContactService	77



114	Diagramma della classe EmergencyContactDTO	78
115	Architettura^G e Flusso di Autenticazione	78
116	Architettura^G moduli Luoghi Sicuri	80
117	Architettura^G moduli Materiale Informativo	81
118	Architettura^G diari	83
119	Architettura^G chatbot	85
120	Architettura^G del sistema di contatti fidati, SOS e Dead man's switch . .	88



1. Introduzione

1.1. Scopo del documento

Il presente documento ha lo scopo di fornire una descrizione completa, formale e strutturata del prodotto software, delineandone in modo rigoroso le caratteristiche tecniche e architetturali.

La **Specifica Tecnica^G** rappresenta il riferimento principale per tutte le attività di progettazione, sviluppo, **Verifica^G** e manutenzione del sistema, assicurando coerenza rispetto ai requisiti definiti nel documento di **Analisi dei requisiti^G** e alle scelte progettuali adottate.

In particolare, il documento si propone di:

- Definire l'**Architettura^G** complessiva del sistema, descrivendo le componenti software, le loro responsabilità e le modalità di interazione attraverso appositi diagrammi
- Specificare i moduli funzionali e le interfacce esposte, includendo i contratti di comunicazione e i formati dei dati scambiati
- Descrivere l'**Architettura^G** di **Deployment^G**, evidenziando la distribuzione delle componenti nei diversi ambienti di esecuzione e le relative dipendenze infrastrutturali
- Documentare le tecnologie, i linguaggi di programmazione, i **Framework^G** e gli strumenti adottati, motivando le scelte effettuate in relazione ai requisiti di progetto e al **Capitolato^G**
- Illustrare i principali design pattern e le soluzioni architetturali utilizzate, con particolare attenzione agli aspetti di scalabilità, manutenibilità e sicurezza
- Fornire indicazioni sui metodi di testing e di **Validazione^G**
- Supportare le attività di evoluzione e manutenzione del software, garantendo una base documentale chiara e coerente.

1.2. Riferimenti

Al fine di evitare ambiguità relative al linguaggio e ai termini utilizzati all'interno dei documenti formali, viene allegato il [Glossario](#). I termini tecnici, gli acronimi e le parole con significato specifico all'interno del progetto sono contrassegnati da una lettera 'G' in apice (es. **Agile^G**) e sono descritti in dettaglio nel documento apposito.



1.2.1 Riferimenti normativi

- **Capitolato^G d'appalto C4: *L'app che Protegge e Trasforma***
 Parti consultate: documento completo
 Versione: 1.0
 Ultimo accesso: 2026-01-14
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C4.pdf>
- **Norme di Progetto**
 Parti consultate: documento completo
 Versione: v.1.0.0
 Ultimo accesso: 2026-02-09
https://swe-bitbybit.github.io/SWE-docs/docs/RTB/documenti_interni/norme_di_progetto.pdf

1.2.2 Riferimenti informativi

- **Slide sulla Progettazione Software del Prof. Vardanega**
 Parti consultate: documento completo
 Versione: A.A.2025/2026
 Ultimo accesso: 2026-03-30
<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/T06.pdf>
- **Materiale relativo alla parte pratica del corso di Ingegneria del Software**
 Parti consultate: documento completo
 Versione: A.A.2022/2023
 Ultimo accesso: 2026-03-30
<https://www.math.unipd.it/~rcardin/swea/2022/>
- **Repository^G Github del Prof. Cardin**
 Ultimo accesso: 2026-03-30
<https://github.com/rcardin/swe-imdb>
- **Guida Flutter sull'Architettura^G di un'applicazione**
 Ultimo accesso: 2026-03-30
<https://docs.flutter.dev/app-architecture/guide>
- **Glossario**
 Parti consultate: documento completo
 Versione: v.1.0.0
 Ultimo accesso: 2026-02-27
https://swe-bitbybit.github.io/SWE-docs/docs/RTB/documenti_interni/Glossario.pdf



2. Tecnologie

Le tecnologie utilizzate sono state scelte trovando un punto d'incontro tra la necessità di garantire la sicurezza e la tutela dei dati personali degli utenti e il bisogno di realizzare un'**Architettura**^G solida, modulare e facilmente estendibile. Di seguito sono riportate tutte le tecnologie adottate

2.1. Framework

2.1.1 Flutter

Flutter è un **Framework**^G open-source sviluppato da Google per la creazione di applicazioni multi-piattaforma a partire da un unico codice sorgente. Consente di sviluppare interfacce per dispositivi mobili, web e desktop utilizzando il linguaggio Dart. Flutter si basa su un sistema di widget componibili, che possono rappresentare qualsiasi elemento dell'interfaccia **Utente**^G. Flutter include un proprio motore grafico (come Impeller o storicamente Skia) che disegna direttamente i componenti a schermo, garantendo elevata coerenza visiva e prestazioni indipendenti dalla piattaforma sottostante. È il **Framework**^G alla base dell'App Che Protegge e Trasforma, il che consente di poter generare build per iOS e Android da un unico codice sorgente, a fronte di un maggiore utilizzo di risorse rispetto alle soluzioni native e di una minore possibilità di controllo diretto e personalizzazione delle funzionalità native della piattaforma.

- **Versione utilizzata:** 3.41.6
- **Documentazione:** <https://docs.flutter.dev/>

2.2. Linguaggi di programmazione

2.2.1 Dart

Dart è un linguaggio di programmazione sviluppato da Google, progettato per la realizzazione di applicazioni client moderne. È un linguaggio tipizzato (con supporto sia statico che dinamico) e orientato agli oggetti. Viene utilizzato principalmente in combinazione con Flutter, dove il codice Dart viene compilato ahead-of-time in codice nativo per migliorare le prestazioni, oppure eseguito tramite compilazione just-in-time durante lo sviluppo. L'utilizzo di Flutter rende di fatto quasi obbligatoria l'adozione di Dart come linguaggio.

- **Versione utilizzata:** 3.11.4
- **Documentazione:** <https://dart.dev/docs>



2.2.2 Python

Python è un linguaggio di programmazione ad alto livello, interpretato e orientato agli oggetti, noto per la sua sintassi semplice e leggibile. Il codice Python viene eseguito da un interprete che traduce le istruzioni in bytecode, poi eseguito dalla macchina virtuale Python. Nel progetto è utilizzato lato **Backend^G** per la scrittura delle funzioni AWS Lambda e per i test tramite Pytest.

- **Versione utilizzata:** 3.14.3
- **Documentazione:** <https://docs.python.org/3/>

2.3. Servizi AWS

2.3.1 Lambda

AWS Lambda è un servizio **Serverless^G** che consente di eseguire codice senza gestire direttamente server o infrastruttura. Le funzioni vengono attivate in risposta a eventi e scalano automaticamente in base al carico. Nel progetto è utilizzato per implementare la logica **Backend^G**, eseguendo funzioni scritte in Python invocate tramite API Gateway.

- **Versione utilizzata:** Runtime Python 3.14
- **Documentazione:** <https://docs.aws.amazon.com/lambda/>

2.3.2 API Gateway

AWS API Gateway è un servizio che permette di creare, pubblicare e gestire **API REST^G** e HTTP, fungendo da punto di ingresso per le richieste client. Gestisce autenticazione, routing e integrazione con altri servizi AWS. Nel progetto è utilizzato per esporre endpoint HTTP che invocano le funzioni Lambda.

- **Versione utilizzata:** HTTP API (v2)
- **Documentazione:** <https://docs.aws.amazon.com/apigateway/>

2.3.3 DynamoDB

Amazon DynamoDB è un database NoSQL completamente gestito, progettato per offrire alte prestazioni e scalabilità automatica. I dati sono memorizzati in tabelle con chiavi primarie e accessibili con bassa latenza. Nel progetto è utilizzato per memorizzare dati strutturati come le note testuali del diario criptato e le chat con il chatbot.

- **Documentazione:** <https://docs.aws.amazon.com/dynamodb/>



2.3.4 S3

Amazon S3 (Simple Storage Service) è un servizio di storage a oggetti che consente di archiviare e recuperare dati in modo scalabile e sicuro. I file sono organizzati in bucket e accessibili tramite API. Nel progetto è utilizzato per memorizzare contenuti multimediali come immagini e file audio.

- **Documentazione:** <https://docs.aws.amazon.com/s3/>

2.3.5 Cognito

Amazon Cognito è un servizio di gestione delle identità che consente autenticazione, autorizzazione e gestione degli utenti. Supporta integrazione con provider esterni come Google. Nel progetto è utilizzato per gestire l'autenticazione degli utenti tramite account Google e per il controllo degli accessi.

- **Documentazione:** <https://docs.aws.amazon.com/cognito/>

2.3.6 Bedrock

Amazon Bedrock è un servizio che consente di utilizzare modelli di intelligenza artificiale generativa tramite API, senza gestire direttamente l'infrastruttura sottostante. Permette l'accesso a diversi modelli foundation per task come generazione di testo. Nel progetto è utilizzato per implementare la funzionalità di chatbot.

- **Documentazione:** <https://docs.aws.amazon.com/bedrock/>

2.3.7 SES

Amazon SES (Simple Email Service) è un servizio per l'invio e la ricezione di email in modo scalabile e affidabile. Supporta integrazione via API e SMTP. Nel progetto è utilizzato per inviare email ai contatti fidati degli utenti.

- **Documentazione:** <https://docs.aws.amazon.com/ses/>

2.3.8 CloudWatch

Amazon CloudWatch è un servizio di monitoraggio e osservabilità che raccoglie metriche, log ed eventi dai servizi AWS. Consente analisi, visualizzazione e configurazione di allarmi. Nel progetto è utilizzato per la raccolta e l'analisi dei log generati dalle funzioni Lambda.

- **Documentazione:** <https://docs.aws.amazon.com/cloudwatch/>



2.3.9 EventBridge

Amazon EventBridge è un servizio di event bus **Serverless^G** che consente di gestire e instradare eventi tra diversi servizi AWS. Esso permette di definire regole per reagire automaticamente a eventi generati da servizi AWS o da fonti esterne. Nel progetto è utilizzato per orchestrare i flussi di controllo di utilizzo dell'app da parte degli utenti, e l'invio delle email secondo le tempistiche stabilite dal Dead man's switch.

- **Documentazione:** <https://docs.aws.amazon.com/eventbridge/>

2.3.10 SAM

AWS **Serverless^G** Application Model (SAM) è un **Framework^G** open-source che semplifica la definizione e il **Deployment^G** di applicazioni **Serverless^G** su AWS. Estende AWS CloudFormation introducendo una sintassi più compatta per risorse come Lambda, API Gateway e DynamoDB e include strumenti per build, testing locale e deploy delle applicazioni. Nel progetto è utilizzato per definire e distribuire l'infrastruttura **Serverless^G** in modo semplificato.

- **Documentazione:** <https://docs.aws.amazon.com/serverless-application-model/>

2.3.11 CloudFormation

AWS CloudFormation è un servizio di **Infrastructure as Code^G** che consente di modellare e gestire risorse AWS tramite template dichiarativi in formato **YAML^G** o JSON. Nel progetto è utilizzato come base per il provisioning dell'infrastruttura, anche tramite template generati da SAM.

- **Documentazione:** <https://docs.aws.amazon.com/cloudformation/>
- **Versioni:** CloudFormation non prevede versioni di servizio esplicite; eventuali aggiornamenti sono gestiti direttamente da AWS. I template possono però utilizzare versioni di specifiche risorse e trasformazioni (es. AWS::**Serverless^G**).

2.4. Librerie

2.4.1 http.dart

http è una libreria per il linguaggio Dart che consente di effettuare richieste HTTP in modo semplice e asincrono. Fornisce metodi per eseguire operazioni come GET, POST, PUT e DELETE, gestendo automaticamente la serializzazione delle richieste e la ricezione delle risposte. È progettata per essere leggera e facilmente integrabile in applicazioni client.

- **Versione utilizzata:** 1.6.0
- **Documentazione:** <https://pub.dev/packages/http>



2.4.2 image_picker.dart

`image_picker` è una libreria Flutter che permette di accedere alla fotocamera e alla galleria del dispositivo per selezionare o acquisire immagini e video. Fornisce un'interfaccia unificata tra piattaforme diverse, gestendo le differenze tra iOS e Android.

- **Versione utilizzata:** 1.2.1
- **Documentazione:** https://pub.dev/packages/image_picker

2.4.3 path_provider.dart

`path_provider` è una libreria Flutter che consente di ottenere i percorsi delle directory del file system del dispositivo, come directory temporanee o documenti applicativi. Facilita l'accesso a percorsi corretti e compatibili tra piattaforme per la gestione dei file.

- **Versione utilizzata:** 2.1.5
- **Documentazione:** https://pub.dev/packages/path_provider

2.5. Tecnologie di analisi statica

2.5.1 SonarQube

SonarQube è una piattaforma open-source per l'analisi continua della qualità del codice. Esegue **Analisi Statica**^G su diversi linguaggi di programmazione per individuare bug, vulnerabilità, code smells e problemi di copertura dei test. Il funzionamento si basa su scanner che analizzano il codice sorgente e inviano i risultati a un server centrale, dove vengono aggregati e visualizzati tramite una dashboard dedicata.

- **Versione utilizzata:** 26.3.0
- **Documentazione:** <https://docs.sonarsource.com/sonarqube/>

2.6. Tecnologie di analisi dinamica

2.6.1 Pytest

Pytest è un **Framework**^G di testing per il linguaggio Python che consente di scrivere test in modo semplice e scalabile. Supporta test funzionali, unitari e di integrazione, offrendo funzionalità come fixture, parametrizzazione dei test e scoperta automatica dei test.

- **Versione utilizzata:** 9.0.2
- **Documentazione:** <https://docs.pytest.org/>



2.6.2 Moto

Moto è una libreria Python che consente di mockare i servizi AWS durante i test. Simula il comportamento delle API AWS, permettendo di testare codice che interagisce con servizi **Cloud^G** senza effettuare chiamate reali. Funziona intercettando le richieste e restituendo risposte simulate coerenti con quelle dei servizi AWS.

- **Versione utilizzata:** 5.1.23
- **Documentazione:** <https://docs.getmoto.org/>

2.7. Altro

2.7.1 Ollama

Ollama è una piattaforma che consente di eseguire e gestire LLM in locale tramite una semplice interfaccia. Permette il download, l'esecuzione e l'interazione con modelli di intelligenza artificiale direttamente in ambiente locale, senza necessità di servizi **Cloud^G** esterni. Il funzionamento si basa su un runtime che gestisce i modelli e espone endpoint per inferenza.

- **Versione utilizzata:** 0.18.3
- **Documentazione:** <https://ollama.com/docs>

2.7.2 Docker

Docker è una piattaforma di containerizzazione che consente di creare, distribuire ed eseguire applicazioni all'interno di ambienti isolati (container). I container includono tutto il necessario per eseguire un'applicazione (codice, runtime, librerie e dipendenze), garantendo portabilità e consistenza tra ambienti diversi. Nel progetto è utilizzato a supporto delle pipeline di **CI/CD^G** e per testare in locale servizi come DynamoDB e S3.

- **Documentazione:** <https://docs.docker.com/>

3. Architettura

3.1. Architettura logica

Il sistema software è progettato secondo i principi di un'**Architettura a Strati^G** (o *Layered Architecture*), che impone una chiara separazione delle responsabilità tra i diversi livelli applicativi del sistema. L'obiettivo cardine di questa scelta è massimizzare l'indipendenza formale della logica di business dalle interfacce grafiche **Utente^G** e dai meccanismi fisici di gestione del dato, favorendo così la facilità di test, la manutenzione nel tempo e il forte disaccoppiamento (*Separation of Concerns^G*).



All'ingresso superiore dell'applicazione si posiziona l'ecosistema logico del dispositivo, mentre le fondamenta sono governate e calcolate dall'infrastruttura di servizio. L'interfaccia tra questi due mondi è ben definita tramite API e totalmente slegata dai linguaggi di programmazione o dai sistemi operativi utilizzati.

3.1.1 Livello Client (Frontend Mobile)

Il client mobile è sviluppato incapsulando il codice secondo i dettami moderni di una **Clean Architecture**^G (fortemente affine al noto pattern architetturale *Model-View-ViewModel* - MVVM). Il codice dell'applicazione è suddiviso in almeno tre strati logici indipendenti e gerarchici:

- **Presentation Layer (View)**: Costituisce il livello più visibile e frontale dell'applicazione. Riceve gli input da parte dell'**Utente**^G e si occupa unicamente di aggregare e renderizzare il layout grafico a schermo tramite widget componenti, senza mai eseguire logica complessa.
- **State Management^G Layer (ViewModel)**: Funge da mediatore tra View e dominio, gestendo lo stato della view. Riceve input dalla view, muta lo stato dell'UI e notifica la view che si aggiornerà sulla base del nuovo stato incapsulato (es. passando da "In caricamento" a "Successo"). Inoltre, gestisce le validazioni elementari dei dati inseriti, comandando ai servizi sottostanti le operazioni formali da compiere.
- **Data Layer^G**: È lo strato più basso dell'ambiente Client, incaricato unicamente dell'acquisizione e gestione dei dati, ed è logicamente suddiviso in *Service* e *Repository*^G. I Service sono gli unici componenti ad interfacciarsi con l'esterno, conoscendo gli endpoint del **Backend**^G per eseguire le chiamate asincrone HTTP e gestendo l'accesso a dati memorizzati localmente o in remoto. I **Repository**^G fungono da intermediari: raccolgono i dati grezzi provenienti dai Service e li formattano in modelli di dominio strutturati (es. un oggetto *Note*), fornendoli al ViewModel puliti e pronti all'uso.

3.1.2 Livello di Servizio (Backend)

L'**Architettura**^G logica adibita all'interazione tra i device mobili e l'ambiente computazionale in **Cloud**^G si fonda su un analogo approccio modulare disaccoppiato in tre sotto-livelli principali:

- **Presentation e Routing Layer**: Costituisce il *Single Point of Entry*^G dell'infrastruttura esposto al software. Ha la pura e unica responsabilità di tracciare e smistare le richieste esterne verso i rami corretti. Intercetta validazioni e autorizzazioni per poi rilasciare formalmente il **Payload**^G agli strati contigui.



- **Business Core / Logic Layer:** Costituisce il cuore elaborativo del sistema, dove risiede l'intera *Business Logic*^G. Questo strato è implementato tramite unità funzionali indipendenti e distribuite (*Funzioni Lambda*) ed è progettato per essere completamente agnostico rispetto ai dettagli di trasporto e comunicazione (come il protocollo HTTP). Il suo compito esclusivo è ricevere i dati già validati dal livello di *Routing*, applicare le regole di dominio, coordinare l'elaborazione degli eventi asincroni e restituire le informazioni elaborate e strutturate.
- **Data Access Layer**^G: Un livello totalmente passivo e disaccoppiato che offre un'astrazione tramite moduli e **Driver**^G. Consente una persistenza dati logica e mascherata ai database sottostanti (quali storage documentali, array non relazionali o conservazione audio/video grezza), celando alla logica di business i layer fisici o i linguaggi di interrogazione diretti della persistenza **Cloud**^G.

3.2. Design pattern utilizzati

3.2.1 Model-view-viewmodel

Descrizione del pattern:

Il pattern Model-view-viewmodel (MVVM) prevede la separazione della parte software responsabile per la resa grafica dalle parti responsabili della **Presentation Logic**^G e **Business Logic**^G in tre componenti principali:

- **Model:** contiene il codice responsabile per la **Business Logic**^G dell'applicazione.
- **View:** contiene la parte responsabile esclusivamente per la resa grafica del software, e delega la gestione dell'interazione **Utente**^G alla componente *ViewModel*.
- **ViewModel:** contiene il codice responsabile per la gestione della **Presentation Logic**^G e per la coordinazione tra i componenti *View* e *Model*.

Ragioni per l'utilizzo:

L'utilizzo del pattern permette una più semplice gestione delle varie parti dell'applicazione, dato che limita notevolmente il coupling tra UI e logica sottostante, migliorandone quindi la manutenibilità, e rende lo sviluppo degli unit test più semplice.

3.2.2 Observer

Descrizione del pattern:

Il pattern Observer definisce la capacità per un oggetto *Publisher* di definire un meccanismo di "iscrizione" per cui un insieme di oggetti *Subscriber* può "osservarlo" ed essere notificato solo in seguito ad un suo cambiamento di stato.

**Ragioni per l'utilizzo:**

L'uso principale del pattern nell'applicazione è nelle classi *ViewModel*, dato che permette di notificare solo i widget che richiedono modifiche dopo un cambiamento di stato del *ViewModel*, invece di dover ricostruire la schermata dopo una qualsiasi interazione, permettendo così una maggiore fluidità dell'interfaccia durante l'uso da parte dell'**Utente^G**. Inoltre il meccanismo di "iscrizione" rimuove la necessità di verifiche frequenti sullo stato del *Publisher* da parte dei *Subscriber* e rende il codice più facilmente estendibile e mantenibile dato che la modifica di una componente non va a influire sulle altre, per esempio, l'aggiunta di un widget alla *View* non richiederebbe modifiche al *ViewModel*.

3.2.3 Proxy**Descrizione del pattern:**

Il pattern Proxy permette la creazione di un oggetto *Proxy* che va a controllare l'accesso all'oggetto reale. L'oggetto *Proxy* deve implementare la stessa interfaccia dell'oggetto reale, in modo da essere sempre utilizzabile quando il codice si aspetta l'oggetto reale, e una volta ricevuta una richiesta deve delegarla ad una istanza dell'oggetto reale.

Ragioni per l'utilizzo:

L'utilizzo del pattern Proxy per le strutture dati delle classi **Chat** e **Note** permette di rimandare la creazione di un loro oggetto fino a quando non effettivamente necessario, evitando così la creazione di tutti gli oggetti e dei loro componenti in una singola volta quando vengono inizialmente caricati, così risparmiando memoria che altrimenti sarebbe sprecata con oggetti pesanti.

3.2.4 Singleton**Descrizione del pattern:**

Il pattern Singleton permette di avere la garanzia che in un dato momento esista al più una singola istanza della classe a cui è applicato. Questo si ottiene facendo in modo che il costruttore dell'oggetto ne crei uno nuovo solamente se non ne esiste già una copia, altrimenti ritorna un puntatore alla copia esistente.

Ragioni per l'utilizzo:

Nell'applicazione questo pattern viene utilizzato per garantire l'esistenza di una sola istanza della classe **DiarySession** in un dato momento, in modo che l'**Utente^G** abbia accesso ad un solo versione del diario, criptato o fittizio, alla volta, evitando la condivisione accidentale dei dati tra le due versioni.



3.3. Architettura di deployment

L'**Architettura^G** di **Deployment^G** del sistema si divide in due componenti principali: l'applicazione Client (eseguita sul dispositivo dell'**Utente^G**) e l'infrastruttura di **Backend^G** (eseguita in **Cloud^G**). Il sistema si basa su un paradigma **Serverless^G** ed **Event-Driven^G**, per garantire scalabilità, elevata disponibilità e abbattimento dei costi legati alla gestione manuale dei server.

3.3.1 Client Mobile

L'applicazione viene rilasciata come eseguibile nativo sui dispositivi mobili degli utenti (Android/iOS). Essendo sviluppata tramite il **Framework^G Flutter**, il codice scritto in Dart viene compilato in formato binario specifico per l'**Architettura^G** hardware del dispositivo ospite. Il Client si interfaccia in modo del tutto indipendente ai servizi di **Backend^G** unicamente tramite la rete Internet, sfruttando protocolli di comunicazione sicuri.

3.3.2 Infrastruttura AWS (Serverless Cloud)

L'intero ecosistema di **Backend^G** è ospitato e gestito all'interno di **Amazon Web Services (AWS)**. Questa scelta architetturale consente:

- **Scalabilità dinamica^G**: le risorse allocate (il calcolo e lo storage) scalano o si accendono/spengono automaticamente in base al volume di richieste ricevute dai Client;
- **Isolamento e Sicurezza**: ogni funzione ha una specifica responsabilità e policy di accesso minima verso le altre risorse, limitando l'esposizione al minimo indispensabile;
- **Gestione del servizio (Zero Ops^G)**: l'uso di infrastrutture completamente gestite e la persistenza dei dati automatizzata tramite **Amazon DynamoDB** e **Amazon S3** libera il team dagli oneri di manutenzione sistemistica.

Il flusso operativo del sistema e la connessione tra i componenti AWS si articolano nei seguenti layer di **Deployment^G**:

- **Amazon API Gateway**: costituisce l'unico punto di ingresso espositivo e si occupa di orchestrare e bilanciare le richieste HTTP provenienti dal Client.
- **Amazon Cognito**: agisce come filtro primario in stretta collaborazione con l'API Gateway, validando i token **Utente^G** e negando le richieste non autorizzate.
- **AWS Lambda**: rappresenta il cuore elaborativo decentralizzato; l'infrastruttura crea dinamicamente container di esecuzione che accolgono le richieste smistate e processano la **Business Logic^G** interna.



- **Amazon DynamoDB e Amazon S3:** compongono lo strato di persistenza profondo ospitando, in alta disponibilità, i dati relazionali/JSON e i contenuti multimediali grezzi sollevando il livello server della necessità di memoria sul disco.

3.3.3 Orchestrazione Event-Driven

Oltre alle comunicazioni sincrone HTTP per le API *REST*, l'**Architettura^G** di **Deployment^G** prevede un'orchestrazione asincrona fondamentale per le funzionalità dell'applicazione (es. le notifiche del sistema *Dead Man's Switch*). I componenti comunicano internamente tramite eventi gestiti da **Amazon EventBridge**, che funge da *Message Bus^G* e permette:

- **Comunicazione asincrona e disaccoppiata:** riducendo la dipendenza operativa tra i servizi;
- L'esecuzione ritardata e pianificata di check sui dati salvati nel database;
- L'instradamento di Eventi critici e allerte verso il servizio **Amazon SES** per l'invio automatizzato di e-mail.

3.3.4 Infrastructure as Code (IaC)

Il **Deployment^G** materiale di tutte le risorse in ambiente AWS non è eseguito manualmente, ma viene automatizzato in modo riproducibile attraverso l'impiego di **AWS SAM** (**Serverless^G Application Model**) e **AWS CloudFormation**. L'infrastruttura logica e fisica è versionata sotto forma di file **YAML^G**, aderendo interamente al paradigma dell'*Infrastructure as Code^G*.

3.4. Architettura Front End

3.4.1 Autenticazione

La funzionalità di autenticazione è stata implementata seguendo il pattern architetturale MVVM raccomandato dalle linee guida di Flutter, adattato al flusso OAuth 2.0 con OpenID Connect descritto nelle specifiche. La struttura separa nettamente le responsabilità tra i layer: il Service gestisce la comunicazione con gli endpoint e il flusso OAuth, il **Repository^G** mantiene lo stato dell'**Utente^G** autenticato e orchestra la traduzione dei dati grezzi in oggetti di dominio tramite i DTO, il ViewModel espone lo stato e le azioni alla View tramite i Command del pattern MVVM. Per la modellazione di un **Utente^G** viene utilizzato il domain model **User**.

L'istanza unica dell'**Utente^G** autenticato viene gestita tramite il Provider di Flutter a livello di app per mantenere una sola istanza attiva per tutta la durata della sessione.

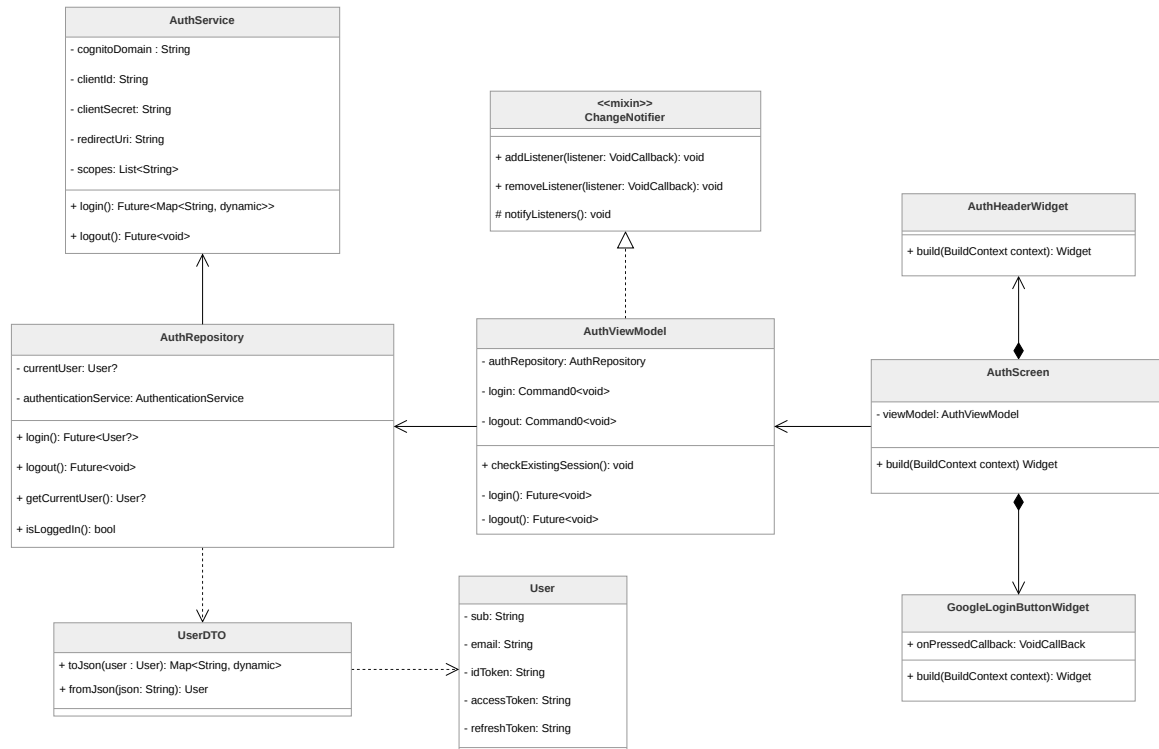
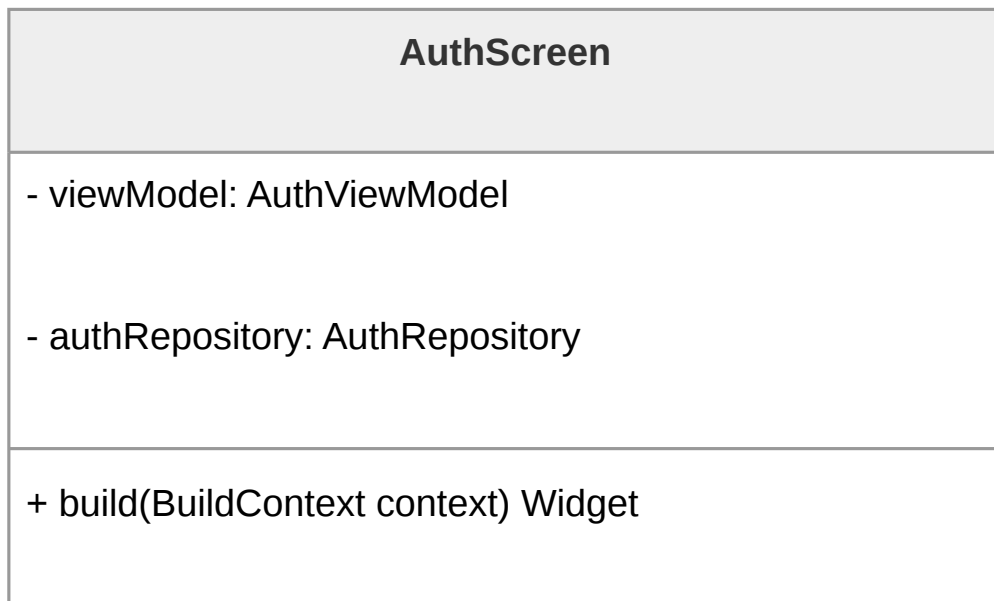


Figura 1: Diagramma delle classi relativo all'autenticazione

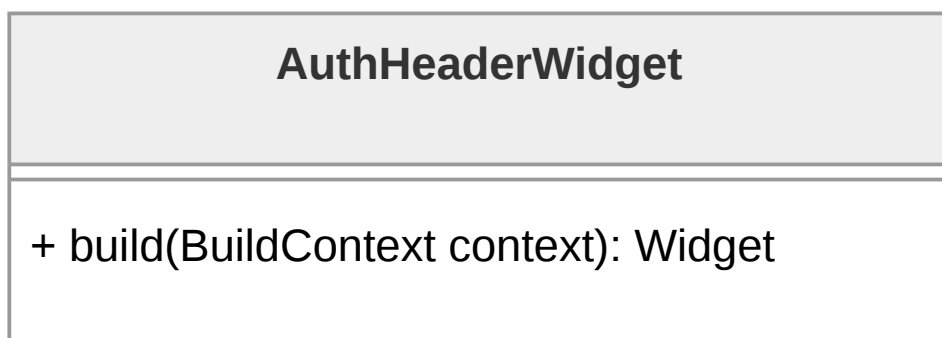
AuthScreen

Widget principale che rappresenta la schermata di autenticazione dell'applicazione. Nel contesto del pattern MVVM, svolge il ruolo di View, occupandosi esclusivamente della costruzione dell'interfaccia Utente tramite il metodo `build(BuildContext context)`. Mantiene un riferimento al `AuthViewModel`, dal quale osserva lo stato dell'autenticazione e a cui delega le azioni dell'Utente^G. La schermata è composta da widget più piccoli come `AuthHeaderWidget` e `GoogleLoginButtonWidget`, favorendo modularità e riusabilità.

Figura 2: Diagramma della classe **AuthScreen**

AuthHeaderWidget

La classe AuthHeaderWidget rientra nel livello View del pattern MVVM e ha il solo compito di costruire la porzione superiore dell'interfaccia, contenente titolo e altri elementi grafici identificativi. Non contiene alcuna logica applicativa e contribuisce alla separazione delle responsabilità tra UI e gestione dello stato.

Figura 3: Diagramma della classe **AuthHeaderWidget**



GoogleLoginButtonWidget

Rappresenta il pulsante che consente all'**Utente^G** di avviare il Processo di autenticazione. È incluso nella **AuthScreen** tramite composizione e riceve una callback **onPressedCallback** che viene eseguita alla pressione del pulsante.

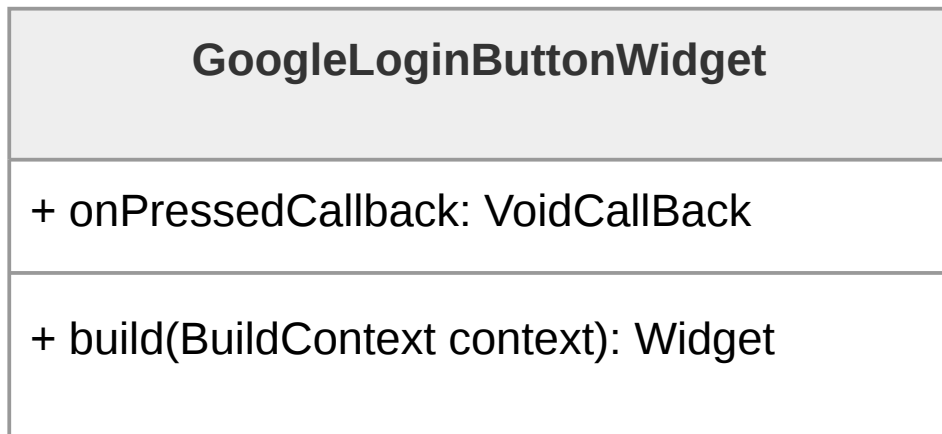


Figura 4: Diagramma della classe **GoogleLoginButtonWidget**

User

Rappresenta il modello di dominio dell'**Utente^G** autenticato all'interno dell'applicazione. Contiene le informazioni essenziali legate all'identità e alla sessione, come l'identificativo univoco e i token di accesso, ed è utilizzata dal resto del sistema per gestire lo stato dell'autenticazione in modo indipendente dal formato dei dati provenienti dall'esterno.

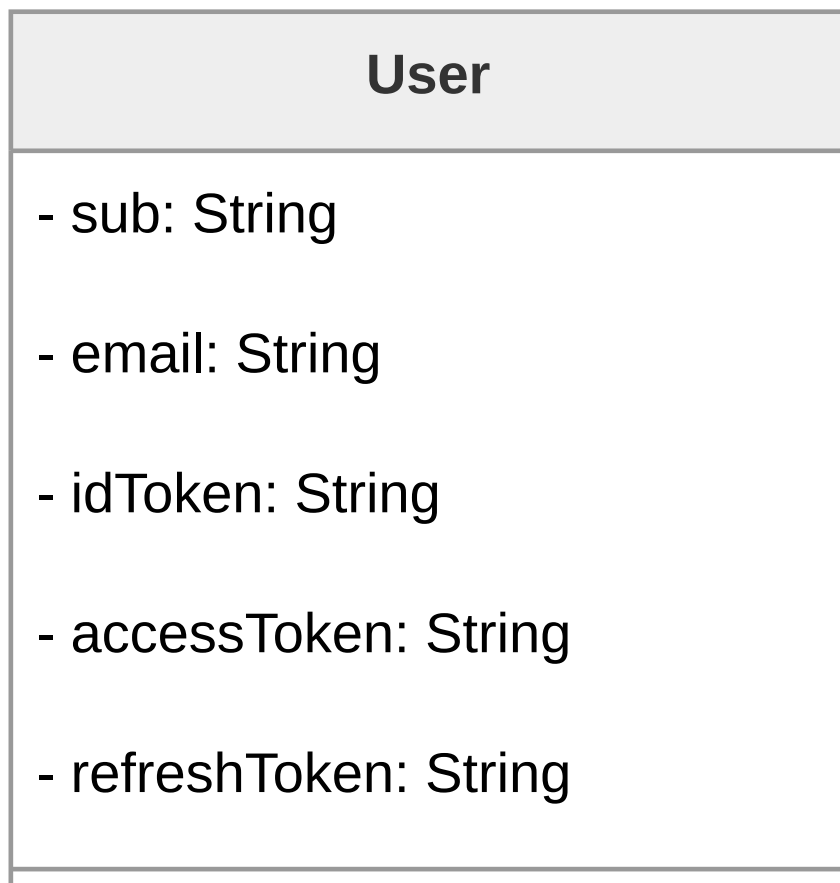
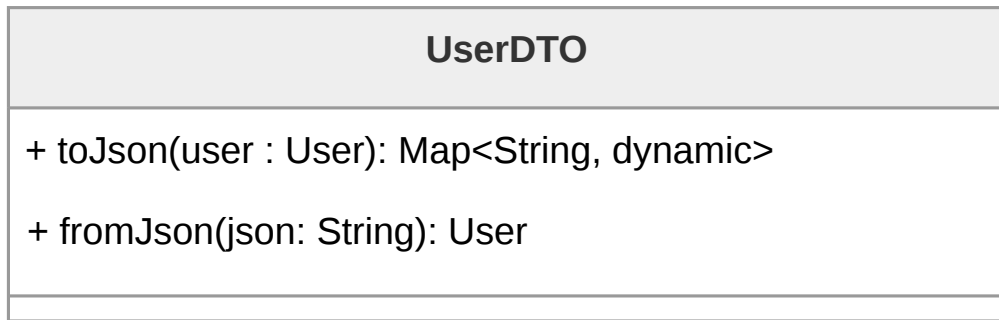


Figura 5: Diagramma della classe **User**

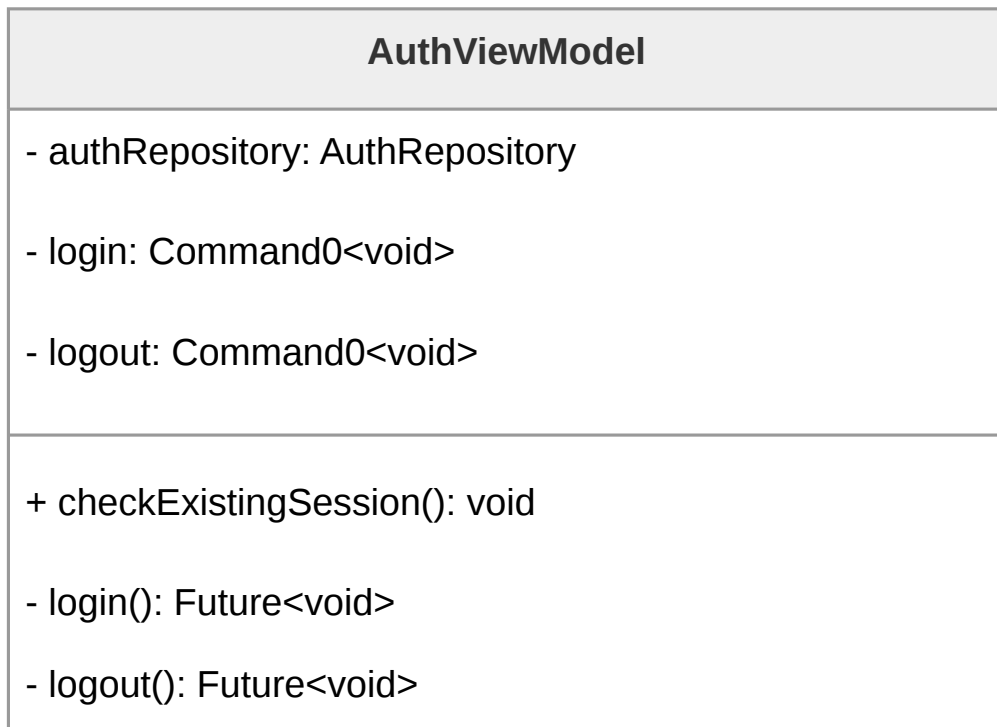
UserDTO

Ha il compito di gestire la conversione tra il modello di dominio **User** e la rappresentazione serializzata dei dati. Attraverso i metodi `toJson` e `fromJson`, consente rispettivamente di trasformare un oggetto **User** in formato JSON e di ricostruirlo a partire da dati serializzati.

Figura 6: Diagramma della classe **UserDTO**

AuthViewModel

Gestisce lo stato dell'autenticazione e coordina le operazioni richieste dalla View interagendo con il **AuthRepository**. Le funzionalità principali, come il login e il logout, sono esposte tramite il Command pattern. La logica concreta è incapsulata nei metodi **login()** e **logout()**, che vengono tipicamente passati come azioni ai rispettivi Command, mentre il metodo **checkExistingSession()** consente di verificare la presenza di una sessione già attiva al momento dell'avvio.

Figura 7: Diagramma della classe **AuthViewModel**

AuthRepository

Funge da intermediario tra il **AuthViewModel** e il livello di accesso ai dati, incapsulando la logica necessaria per gestire l'autenticazione. Mantiene lo stato dell'**Utente^G** corrente tramite **currentUser** e delega le operazioni di comunicazione remota a **AuthenticationService**. Il **Repository^G** espone metodi per effettuare il login, il logout, recuperare l'**Utente^G** corrente e verificare se esiste una sessione attiva.

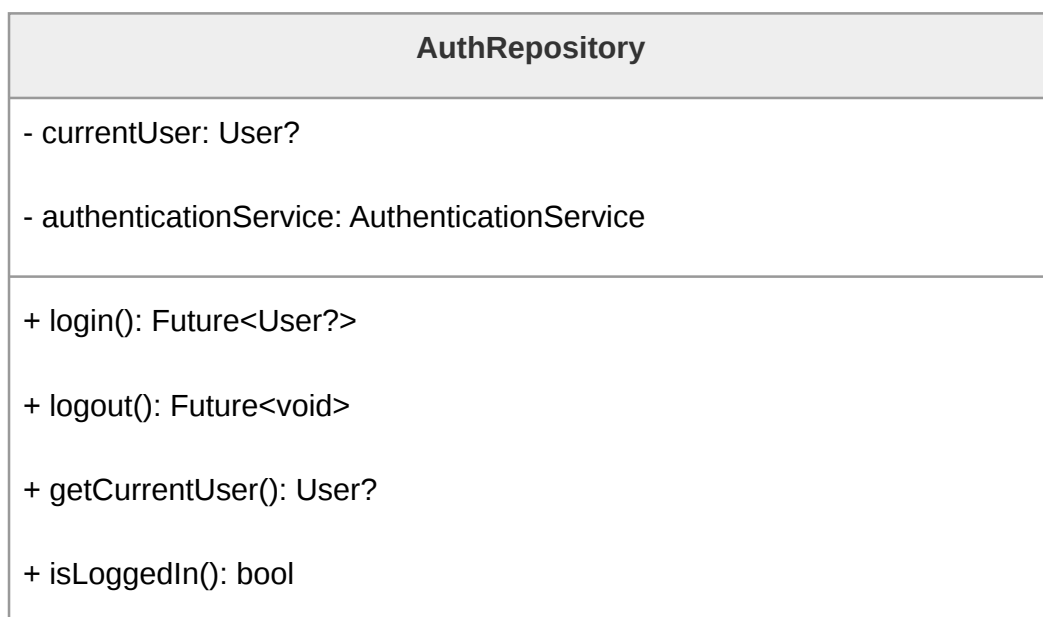


Figura 8: Diagramma della classe **AuthRepository**

AuthService

È responsabile della comunicazione con il sistema di autenticazione esterno, ad esempio un provider basato su OAuth o AWS Cognito. Contiene le informazioni di configurazione necessarie, tra cui dominio, credenziali del client, URI di redirect e scope richiesti. Si occupa esclusivamente di eseguire le chiamate di rete per il login e il logout

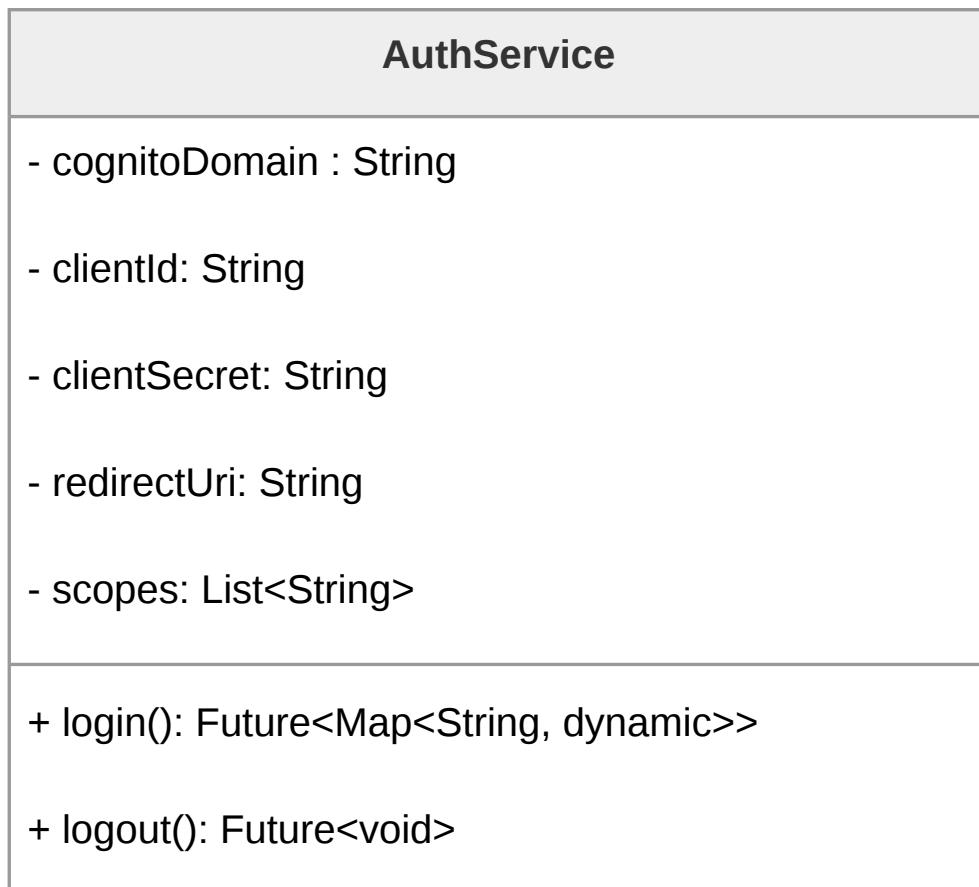


Figura 9: Diagramma della classe **AuthenticationService**

3.4.2 Invio SOS

La funzionalità di invio dell>alert SOS è strutturata seguendo il pattern MVVM raccomandato dalla guida ufficiale di Flutter, ed utilizza anch'essa il pattern Command. Il diagramma descrive l'insieme delle classi che collaborano per permettere all'**Utente^G** di inviare un messaggio di soccorso ai propri contatti fidati tramite la pressione di un pulsante nel **SosBottomSheetWidget**, visibile in tutte le schermate dell'applicazione. La classe **SosAlertViewModel** coordina le precondizioni necessarie all'invio, verificando la presenza di contatti fidati tramite le informazioni contenute nel **TrustedContactRepository**, prima di inoltrare l'operazione al **SosAlertRepository**.

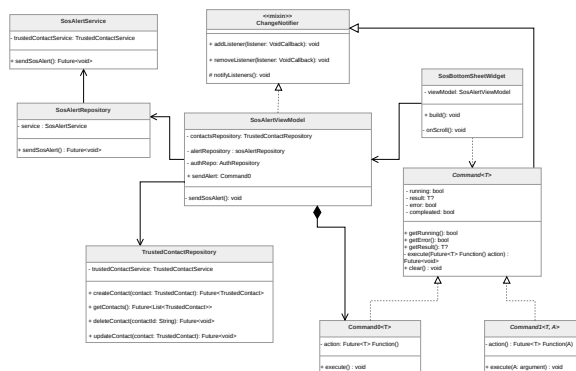


Figura 10: Diagramma delle classi relativo alla funzionalità dell'invio email di soccorso

SosBottomSheetWidget

Widget che funge da punto di ingresso visivo per la funzionalità. Viene mostrato e nascosto in risposta allo scroll dell'**Utente^G** tramite `_onScroll()` ed è presente in tutte le schermate dell'applicazione. Mantiene un riferimento al **SosAlertViewModel** da cui riceve lo stato corrente dell'operazione e a cui delega l'esecuzione dell'invio quando l'**Utente^G** preme il pulsante.

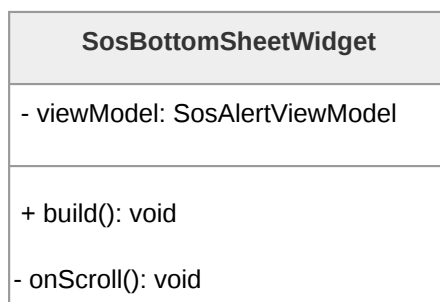


Figura 11: Diagramma della classe **SosBottomSheetWidget**

SosAlertViewModel

Layer di presentazione della funzionalità. Coordina le dipendenze necessarie all'invio dei segnali d'allarme:

- **TrustedContactRepository** per la verifica della presenza di contatti;
- **SosAlertRepository** per l'invio effettivo;
- **AuthRepository** per recuperare l'identità dell'**Utente^G** autenticato.

Espone `sendAlert` come **Command0** che la View può eseguire direttamente. La logica di orchestrazione delle precondizioni è contenuta nel metodo privato `_sendSosAlert()`, che viene passato come action al Command nel costruttore.

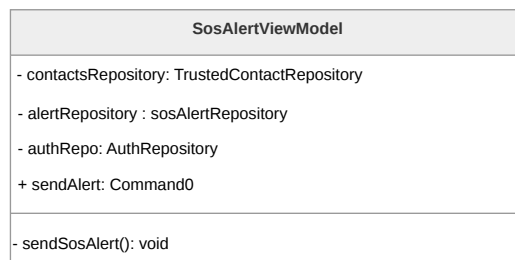


Figura 12: Diagramma della classe **SosAlertViewModel**

SosAlertRepository

Ha la singola responsabilità di orchestrare l'invio dell>alert SOS. Delega la chiamata HTTP al **Backend^G** tramite **SosAlertService** e rappresenta il punto di separazione tra il layer di presentazione e il layer di accesso ai dati per questa funzionalità specifica.

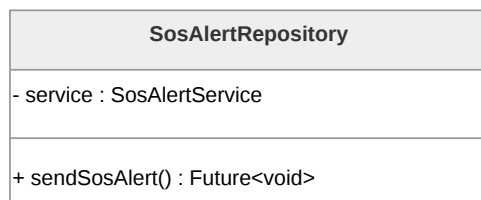


Figura 13: Diagramma della classe **SosAlertRepository**

SosAlertService

Gestisce la comunicazione con l'endpoint dedicato all'invio dell>alert. Riceve da **SosAlertRepository** la richiesta di invio e si occupa esclusivamente della chiamata di rete, restituendo i dati grezzi della risposta.

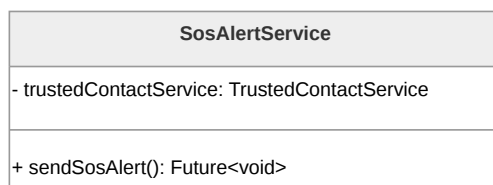


Figura 14: Diagramma della classe **SosAlertService**

Command<T>

È la classe base astratta del pattern. Mantiene internamente lo stato dell'azione tramite i campi privati **_running**, **_result** e **_error**. Espone queste informazioni tramite getter pubblici che la View utilizza per aggiornare l'interfaccia. Il metodo privato **_execute()**



contiene la logica comune a tutte le implementazioni concrete: impedisce esecuzioni multiple simultanee, aggiorna lo stato prima e dopo l'esecuzione dell'azione, e gestisce le eccezioni.

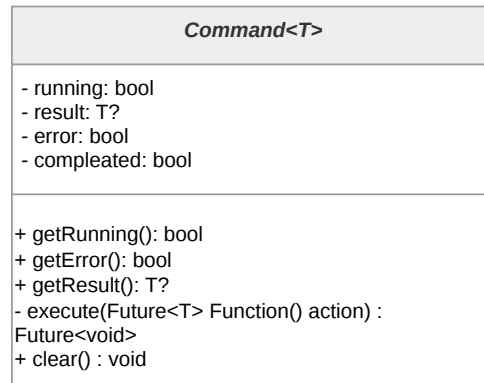


Figura 15: Diagramma della classe **Command<T>**

Command0<T>

È l'implementazione concreta di **Command<T>** per azioni senza argomenti. Riceve nel costruttore una funzione di tipo **Future<T> Function()** che rappresenta l'azione da eseguire. Espone il metodo pubblico **execute()** che delega a **_execute()** della classe base passando l'azione ricevuta nel costruttore.

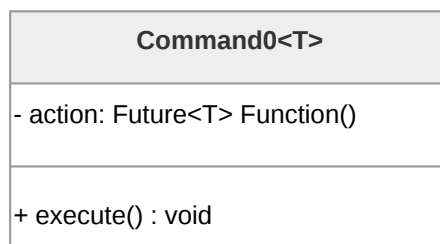


Figura 16: Diagramma della classe **Command0<T>**

Command1<T, A>

È l'implementazione concreta di **Command<T>** per azioni che richiedono un argomento di tipo A. Riceve nel costruttore una funzione di tipo **Future<T> Function(A)** e il metodo pubblico **execute()** accetta l'argomento che viene passato all'azione al momento dell'esecuzione.

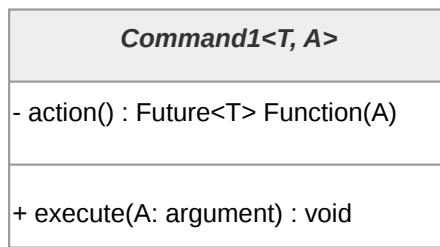


Figura 17: Diagramma della classe **Command1<T, A>**

3.4.3 Chatbot

La funzionalità del chatbot permette all'**Utente^G** di comunicare via chat con un modello di intelligenza artificiale (LLM) e di memorizzare e caricare tutte le chat passate. La funzionalità del chatbot è implementata secondo il pattern MVVM e le linee guida architetturali di Flutter, distinguendo quindi un **Data Layer^G**, composto da Service e **Repository^G**, e un **UI Layer^G**, composto da View e ViewModel. Tra questi layer figurano i vari domain models: le classi Chat (**ProxyChat** e **LocalChat**), **ChatMessage** e **MessageResponse**.



La classe `ChatbotScreen` rappresenta la schermata dell'applicazione dedicata al Chatbot all'interno del *Presentation Layer*. La classe è composta a sua volta da numerosi widget e contiene la minima logica necessaria al rendering di tali componenti visivi, permettendo quindi una netta separazione tra presentazione e logica secondo il pattern *MVVM*. Tra i numerosi widget che la costituiscono, i principali sono `ChatWidget`, `ChatHistoryWidget`, `ChatbotModeToggleWidget` e `ChatbotSendMessageWidget`.





ChatWidget

La classe **ChatWidget** è responsabile della visualizzazione della conversazione attiva. In quanto **Consumer**, essa osserva il **ChatbotViewModel** per ottenere i dati aggiornati e reagisce ai cambiamenti dello stato applicativo, garantendo la costruzione automatica dei widget rappresentanti i messaggi scambiati in chat.

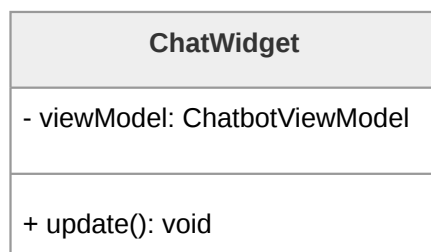


Figura 20: Diagramma della classe **ChatWidget**

ChatHistoryWidget

ChatHistoryWidget si occupa di mostrare l'elenco delle conversazioni passate sotto forma di anteprime. Tale widget è inserito all'interno di un menu a tendina dedicato, e, in quanto **Consumer**, osserva il **ChatbotViewModel** per recuperare i dati aggiornati relativi alle chat passate. Questo widget trae grande beneficio dall'utilizzo del pattern Proxy per il fetch delle chat. Tale pattern consente infatti di generare la lista di anteprime delle conversazioni passate senza dover recuperare l'intero contenuto di ogni singola chat.

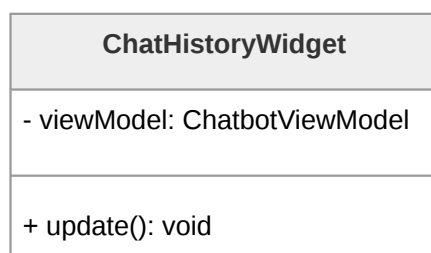


Figura 21: Diagramma della classe **ChatHistoryWidget**

ChatbotModeToggleWidget

Questo componente dell'interfaccia consente all'**Utente^G** di selezionare la modalità operativa del chatbot. Le due modalità disponibili sono il Detective delle Relazioni e lo



Specchio Intelligente. La modalità in uso viene specificata nell'invocazione dei metodi relativi all'invio di un messaggio in chat e determinano come il chatbot risponde ai messaggi dell'**Utente^G**. La generazione della risposta è a completo carico del **Backend^G**.

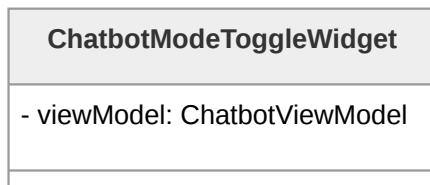


Figura 22: Diagramma della classe **ChatbotModeToggleWidget**

ChatbotSendMessageWidget

ChatbotSendMessageWidget è un pulsante che gestisce l'invio dei messaggi da parte dell'**Utente^G**, delegando la gestione dei nuovi messaggi al ViewModel. Quando **ChatbotSendMessageWidget** viene premuto, il messaggio contenuto nel campo d'inserimento testuale viene inoltrato a **ChatbotViewModel**, che a sua volta comunica con **ChatbotRepository** in modo da memorizzare il messaggio nella chat corrente e di ricevere la risposta dal LLM.

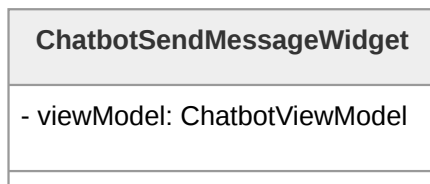


Figura 23: Diagramma della classe **ChatbotSendMessageWidget**

ChatbotCreateChatWidget

Questo widget fornisce all'**Utente^G** la possibilità di avviare una nuova conversazione. Anche in questo caso, la logica viene delegata al ViewModel, preservando il disaccoppiamento tra interfaccia e dominio applicativo.

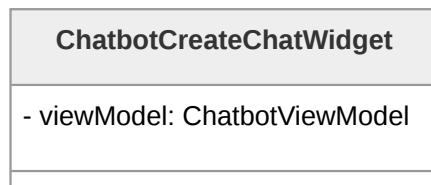


Figura 24: Diagramma della classe **ChatbotCreateChatWidget**

ChatbotViewModel

ChatbotViewModel gestisce lo stato dell'interfaccia chatbot: mantiene la lista delle conversazioni passate, la chat correntemente aperta e la modalità operativa selezionata. In quanto **ChangeNotifier**, notifica automaticamente le classi View (**ChatWidget**, **ChatHistoryWidget** e i widget di azione) ogni volta che lo stato cambia. Per accedere ai dati si affida a **ChatbotRepository**, al quale delega il recupero e la persistenza delle conversazioni.

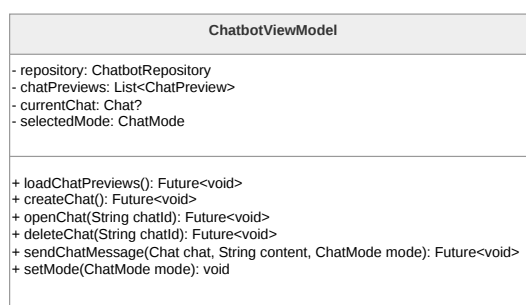
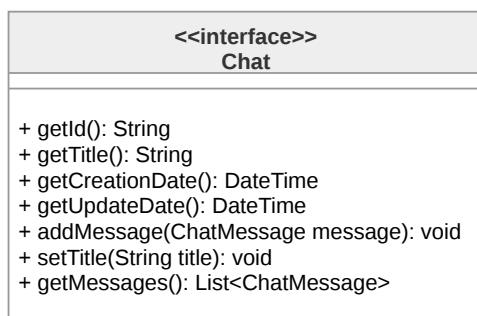


Figura 25: Diagramma della classe **ChatbotViewModel**

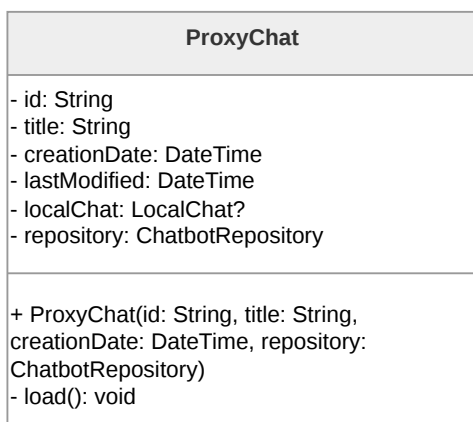
Chat

L'interfaccia **Chat** definisce il modello astratto di una conversazione tra **Utente^G** e il chatbot. Insieme a **ProxyChat** e **LocalChat**, è utilizzata nell'implementazione del pattern Proxy.

Figura 26: Diagramma della classe **Chat**

ProxyChat

ProxyChat implementa l'interfaccia **Chat** e viene utilizzata da **ChatbotViewModel** per rappresentare le conversazioni storiche. Ritarda il caricamento completo dei messaggi, delegandolo a **ChatbotRepository** solo quando necessario, e mantiene internamente un riferimento a **LocalChat**, che istanzia e popola al primo accesso effettivo ai dati.

Figura 27: Diagramma della classe **ProxyChat**

LocalChat

LocalChat è l'implementazione del modello di conversazione che mantiene in memoria tutte le informazioni e i messaggi associati alla chat. Essa rappresenta l'oggetto reale utilizzato da **ProxyChat** quando c'è effettivamente bisogno di caricare i messaggi della chat, ad esempio per visualizzarli a schermo.

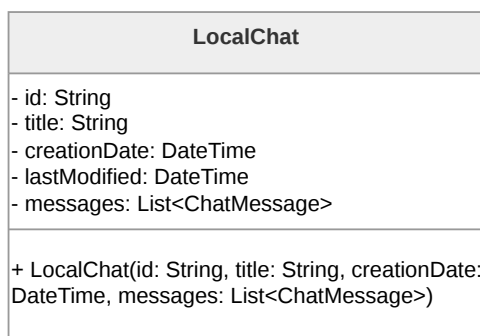


Figura 28: Diagramma della classe **LocalChat**

ChatMessage

ChatMessage modella un singolo messaggio all'interno di una conversazione, includendo contenuto, tipologia e data di creazione e di aggiornamento. Fornisce inoltre un modo per distinguere tra messaggi generati dall'**Utente^G** e quelli prodotti dal LLM.

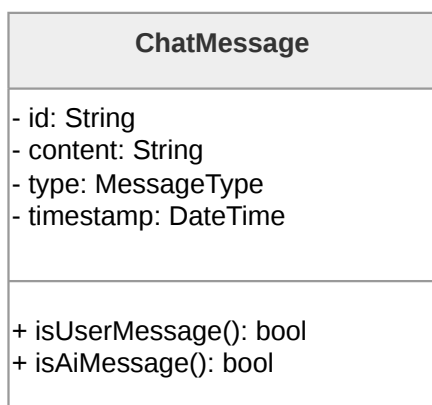
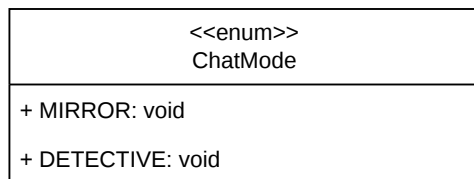


Figura 29: Diagramma della classe **ChatMessage**

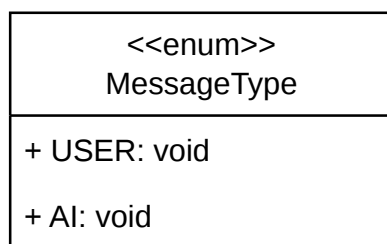
ChatMode

ChatMode è un'enumerazione che rappresenta le due diverse modalità operative del chatbot. Viene utilizzato da **ChatbotViewModel**, che lo inoltra a **ChatbotRepository**, il quale lo utilizza per invocare i metodi del **ChatbotService** relativi all'invio dei messaggi in chat. Sarà poi il **Backend^G** a interpretarlo e utilizzarlo per generare la risposta al messaggio inviato dall'**Utente^G**.

Figura 30: Diagramma della classe **ChatMode**

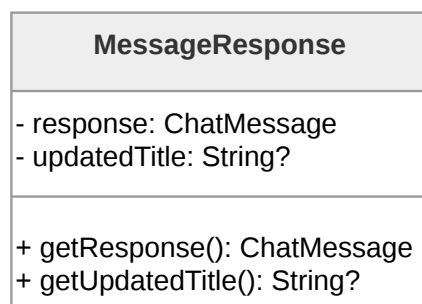
MessageType

MessageType è un'enumerazione che consente di classificare i messaggi in base alla loro origine, distinguendo tra input dell'**Utente^G** e output del LLM. È utilizzata nella classe **ChatMessage** e necessaria per stabilire l'impaginazione dei messaggi all'interno di **ChatWidget**.

Figura 31: Diagramma della classe **MessageType**

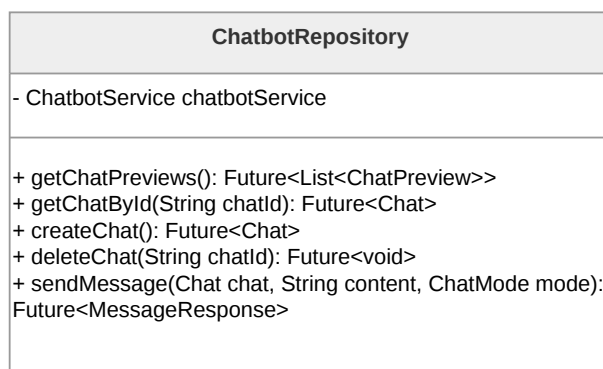
MessageResponse

MessageResponse incapsula il risultato di un'interazione con il LLM, includendo il messaggio generato e, eventualmente, il titolo generato a partire dal messaggio originale. È utilizzato da **ChatbotRepository**, che a sua volta lo ritorna in risposta a **ChatbotViewModel** per l'aggiornamento dello stato della View.

Figura 32: Diagramma della classe **MessageResponse**

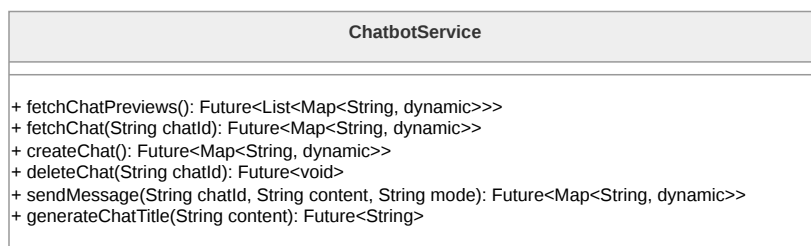
ChatbotRepository

ChatbotRepository è il tramite tra **ChatbotViewModel** e la comunicazione di rete: riceve le richieste dal ViewModel, le inoltra a **ChatbotService** e trasforma le risposte grezze in oggetti di dominio (**LocalChat**, **ChatMessage**) tramite **ChatDTO**, restituendoli al ViewModel in un formato pronto all'uso.

Figura 33: Diagramma della classe **ChatbotRepository**

ChatbotService

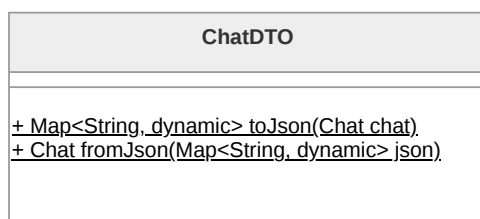
ChatbotService rappresenta il livello infrastrutturale responsabile della comunicazione con le API remote. Si occupa della gestione delle richieste e delle risposte, operando su rappresentazioni dei dati adatte allo scambio su rete. Restituisce i dati grezzi a **ChatbotRepository**, il quale si occupa poi di trasformarli in modelli di dominio.

Figura 34: Diagramma della classe **ChatbotService**

ChatDTO

ChatDTO è un oggetto di trasferimento dati utilizzato per convertire le informazioni tra il formato utilizzato dal sistema esterno e quello del dominio applicativo. Ha due ruoli principali:

- trasformare i risultati delle chiamate ad API da JSON a oggetti di tipo **Chat**
- serializzare oggetti di tipo **Chat**, generando la loro rappresentazione JSON per l'invio alle API.

Figura 35: Diagramma della classe **ChatDTO**



3.4.4 Diario criptato e diario fittizio

La funzionalità di diario criptato e diario fittizio permette all'**Utente^G** di utilizzare due archivi separati in cui memorizzare note, contenenti testo, immagini e/o contenuti audio, protetti da password. L'implementazione della funzionalità segue il pattern MVVM e le linee guida del **Framework^G** Flutter, risultanti nella suddivisione tra un **Data Layer^G**, composto da classi Service e **Repository^G**, responsabili per la **Business Logic^G** e per la **Persistence Logic^G** rispettivamente, ed un **UI Layer^G**, composto da classi View e Viewmodel. Gli oggetti principali della funzionalità sono le classi Note (**ProxyNote** e **LocalNote**), **NoteElement** e **DiarySession**.

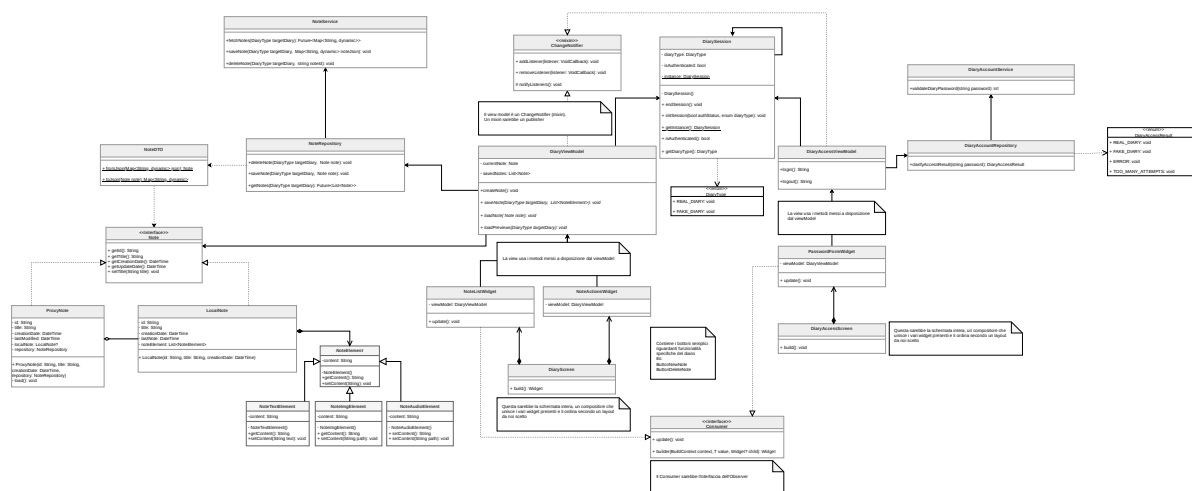


Figura 36: Diagramma delle classi relativo alle funzionalità del diario criptato



DiaryAccessScreen

La classe **DiaryAccessScreen** si occupa della rappresentazione visiva della schermata per l'accesso alla funzionalità dei diari. Contiene solamente la logica necessaria per il rendering dei widget che la compongono, tra cui **PasswordFormWidget**, per garantire la separazione tra presentazione e logica, come previsto dal pattern MVVM. Per questo delega la gestione dello stato a **DiaryAccessViewModel**.

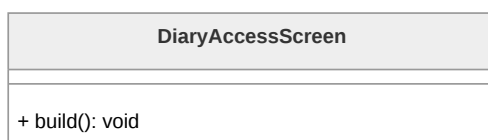


Figura 37: Diagramma della classe **DiaryAccessScreen**

PasswordFormWidget

La classe **PasswordFormWidget** è responsabile della visualizzazione dei campi utilizzati per l'accesso ai diari. **PasswordFormWidget** è un **Consumer** che osserva **DiaryAccessViewModel** per ottenere i dati aggiornati e reagire di conseguenza.

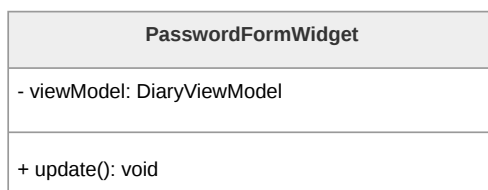
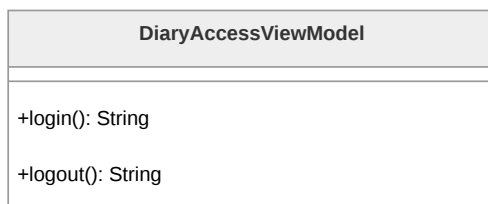


Figura 38: Diagramma della classe **PasswordFormWidget**

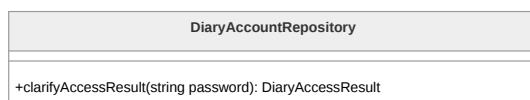
DiaryAccessViewModel

La classe **DiaryAccessViewModel** gestisce lo stato dell'applicazione e coordina le operazioni tra i Layer, implementando il pattern Observer. la classe è un **ChangeNotifier** e mantiene una lista di *listeners* che vengono notificati quando avvengono cambiamenti che necessitano di un loro aggiornamento, in questo caso componenti che devono reagire a tentativi di accesso ai diari da parte dell'**Utente^G**. Per accedere ai dati e verificare le credenziali si affida alla classe **DiaryAccountRepository**.

Figura 39: Diagramma della classe **DiaryAccessViewModel**

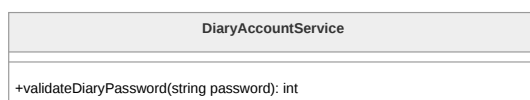
DiaryAccountRepository

DiaryAccountRepository si occupa della trasformazione delle risposte ricevute da **DiaryAccountService** in oggetti di dominio e nascondendone la natura al resto dell'applicazione, fungendo da strato di astrazione tra il *service* ed il ViewModel.

Figura 40: Diagramma della classe **DiaryAccountRepository**

DiaryAccountService

La classe **DiaryAccountService** rappresenta lo strato infrastrutturale responsabile della comunicazione con le API remote, occupandosi della gestione delle richieste e delle risposte. Restituisce i dati grezzi a **DiaryAccountRepository**, il quale si occupa poi di trasformarli in oggetti di dominio.

Figura 41: Diagramma della classe **DiaryAccountService**

DiaryAccessResult

L'enumerazione **DiaryAccessResult** permette la classificazione della risposta ad un tentativo di accesso ai diari da parte dell'**Utente^G**, distinguendo tra accesso avvenuto con successo ad uno dei due tipi di diario, e accesso fallito con errore o superamento del limite di tentativi di accesso.

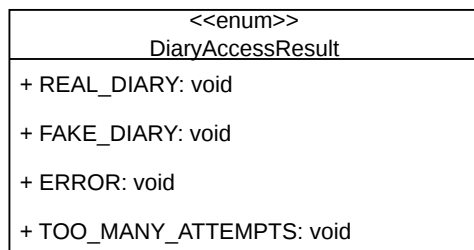


Figura 42: Diagramma della classe **DiaryAccessResult**

DiaryScreen

La classe **DiaryScreen** gestisce la rappresentazione visiva della schermata principale dei diari nella *presentation layer*, contenendo solamente la logica necessaria al rendering dei suoi componenti widget, tra cui **NoteListWidget** e **NoteActionsWidget**. La gestione dello stato è delegata alla classe **DiaryViewModel**.

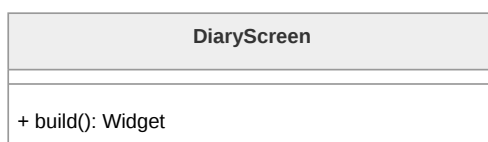


Figura 43: Diagramma della classe **DiaryScreen**

NoteListWidget

La classe **NoteListWidget** si occupa della visualizzazione della lista delle note contenute nel diario in cui l'**Utente^G** ha effettuato l'accesso e l'eventuale visualizzazione dei contenuti di una nota selezionata. Trattandosi di un **Consumer**, la classe osserva gli aggiornamenti dei dati in **DiaryViewModel** in modo da reagire di conseguenza alla selezione, modifica o eliminazione delle note.

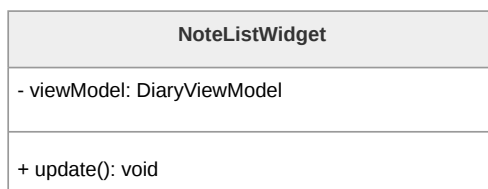


Figura 44: Diagramma della classe **NoteListWidget**

NoteActionsWidget

La classe **NoteActionsWidget** si occupa della visualizzazione delle componenti di interazione della funzionalità di diario. La logica applicativa viene delegata a **DiaryViewModel**, preservando il disaccoppiamento tra interfaccia e dominio applicativo.

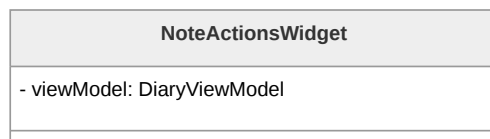


Figura 45: Diagramma della classe **NoteActionsWidget**

DiaryViewModel

DiaryViewModel si occupa della gestione dello stato dell'applicazione e del coordinamento delle operazioni tra UI e **Data Layer^G**. La classe tiene conto della lista di note presenti all'interno del diario attualmente aperto e della nota attualmente aperta dall'**Utente^G**. Inoltre delega alla classe **NoteRepository** l'accesso e la gestione dei dati delle note per le operazioni di memoria. La classe implementa il Mixin **ChangeNotifier**, e quindi il pattern Observer, mantenendo una lista di *listeners* che vengono notificati a seguito di operazioni a loro rilevanti, in questo caso **NoteListWidget** e **NoteActionsWidget**.

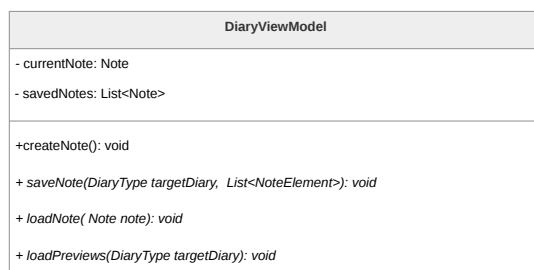


Figura 46: Diagramma della classe **DiaryViewModel**

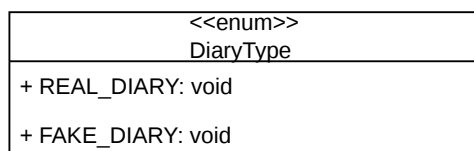
DiarySession

La classe **DiarySession** modella l'istanza effettiva del diario, tenendo traccia del tipo di diario a cui l'**Utente^G** ha effettuato l'accesso e gestendo le informazioni di sessione. La classe implementa il pattern Singleton per fare in modo che in ogni dato momento di utilizzo dell'applicazione possa esistere al più una singola istanza di diario.

Figura 47: Diagramma della classe **DiarySession**

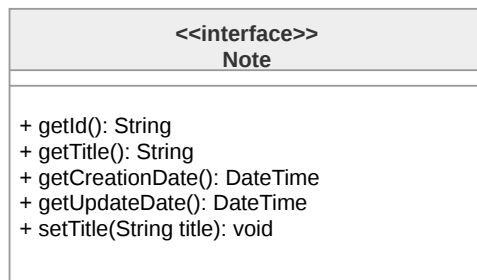
DiaryType

L'enumerazione **DiaryType** rappresenta le due tipologie di diario disponibili e viene utilizzata da **DiarySession** per comunicare a **DiaryViewModel** quale archivio di note visualizzare.

Figura 48: Diagramma della classe **DiaryType**

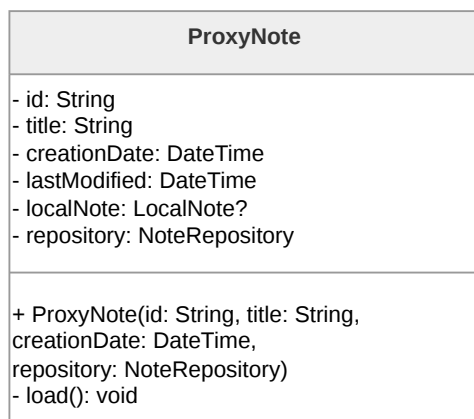
Note

L'interfaccia **Note** definisce i metodi necessari per gli oggetti rappresentanti le note all'interno dei diari. Implementa il pattern Proxy insieme alle classi **ProxyNote** e **LocalNote**.

Figura 49: Diagramma della classe **Note**

ProxyNote

La classe **ProxyNote** realizza il pattern Proxy per rendere più efficiente il caricamento da **Repository^G** delle note, permettendo di rimandare il caricamento dei dati interni di una nota fino a quando non è effettivamente necessario, e così risparmiando risorse di sistema. La classe contiene un riferimento ad un oggetto **LocalNote** a cui delega le operazioni una volta effettuato il caricamento della nota.

Figura 50: Diagramma della classe **ProxyNote**

LocalNote

LocalNote è l'implementazione dell'interfaccia **Note** che rappresenta l'oggetto reale, utilizzato dall'implementazione proxy per le operazioni sulla nota una volta effettuato il suo caricamento.

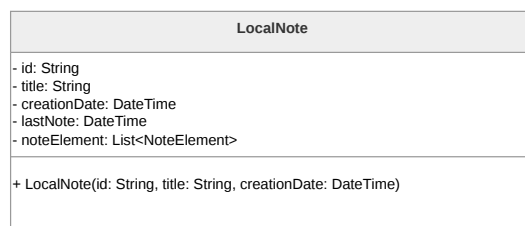


Figura 51: Diagramma della classe **LocalNote**

NoteElement

La classe astratta **NoteElement** rappresenta un elemento presente all'interno di una nota e ha delle specializzazioni in base al tipo di contenuto dell'oggetto, che può essere testuale, un'immagine oppure del contenuto audio.

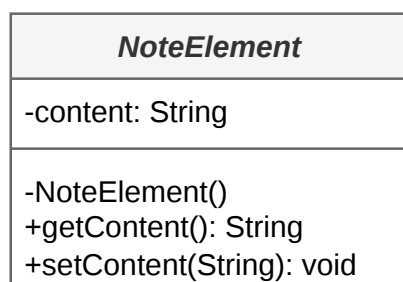


Figura 52: Diagramma della classe **NoteElement**

NoteTextElement

La classe concreta **NoteTextElement** è la specializzazione di **NoteElement** per contenuti testuali.

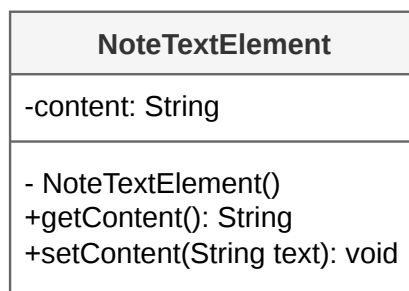


Figura 53: Diagramma della classe **NoteTextElement**



NoteImgElement

La classe concreta **NoteImgElement** è la specializzazione di **NoteElement** per contenuti di tipo grafico.

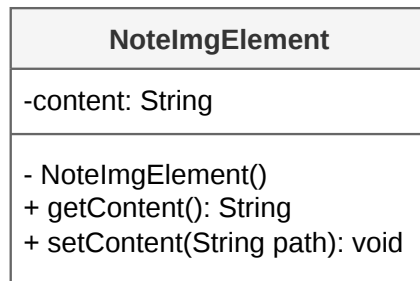


Figura 54: Diagramma della classe **NoteImgElement**

NoteAudioElement

La classe concreta **NoteAudioElement** è la specializzazione di **NoteElement** per contenuti audio.

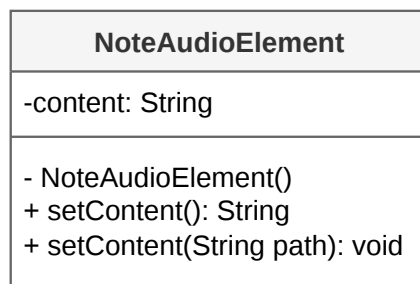


Figura 55: Diagramma della classe **NoteAudioElement**

NoteService

La classe **NoteService** rappresenta il livello infrastrutturale dedicato alla comunicazione con le API remote, gestendo richieste e risposte durante le operazioni sulle note. Restituisce i dati grezzi a **NoteRepository**, il quale si occupa poi di trasformarli in modelli di dominio tramite **NoteDTO**.

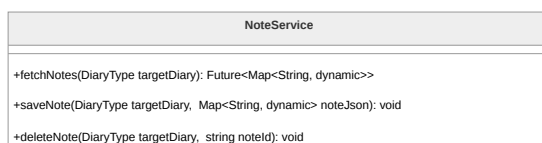


Figura 56: Diagramma della classe **NoteService**

NoteRepository

La classe **NoteRepository** funge da strato di astrazione tra **DiaryViewModel** e la **Persistence Logic^G** di **NoteService**, trasformando i dati grezzi in oggetti di dominio tramite **NoteDTO** e nascondendone la natura al resto dell'applicazione.

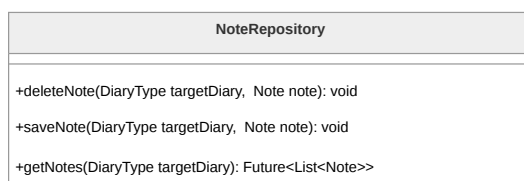


Figura 57: Diagramma della classe **NoteRepository**

NoteDTO

NoteDTO è un Data Transfer Object con due ruoli principali:

- deserializzare i risultati delle chiamate API, in formato JSON, trasformandoli in oggetti di tipo **Note**.
- serializzare oggetti di tipo **Note**, generando la loro rappresentazione JSON per operazioni che richiedono l'invio dei dati alle API, come il salvataggio di una nuova nota.

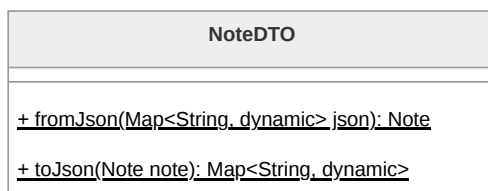


Figura 58: Diagramma della classe **NoteDTO**

3.4.5 Contatti Fidati

La funzionalità dei Contatti Fidati permette all'**Utente^G** di gestire una lista di persone di fiducia da avvisare in caso di inattività tramite il sistema di *Dead Man's Switch*.



Analogamente al resto dell'applicazione, la funzionalità è implementata secondo il pattern MVVM e le linee guida architetturali di Flutter, distinguendo un **Data Layer^G**, composto da Service e **Repository^G**, e un **UI Layer^G**, composto da View e ViewModel.

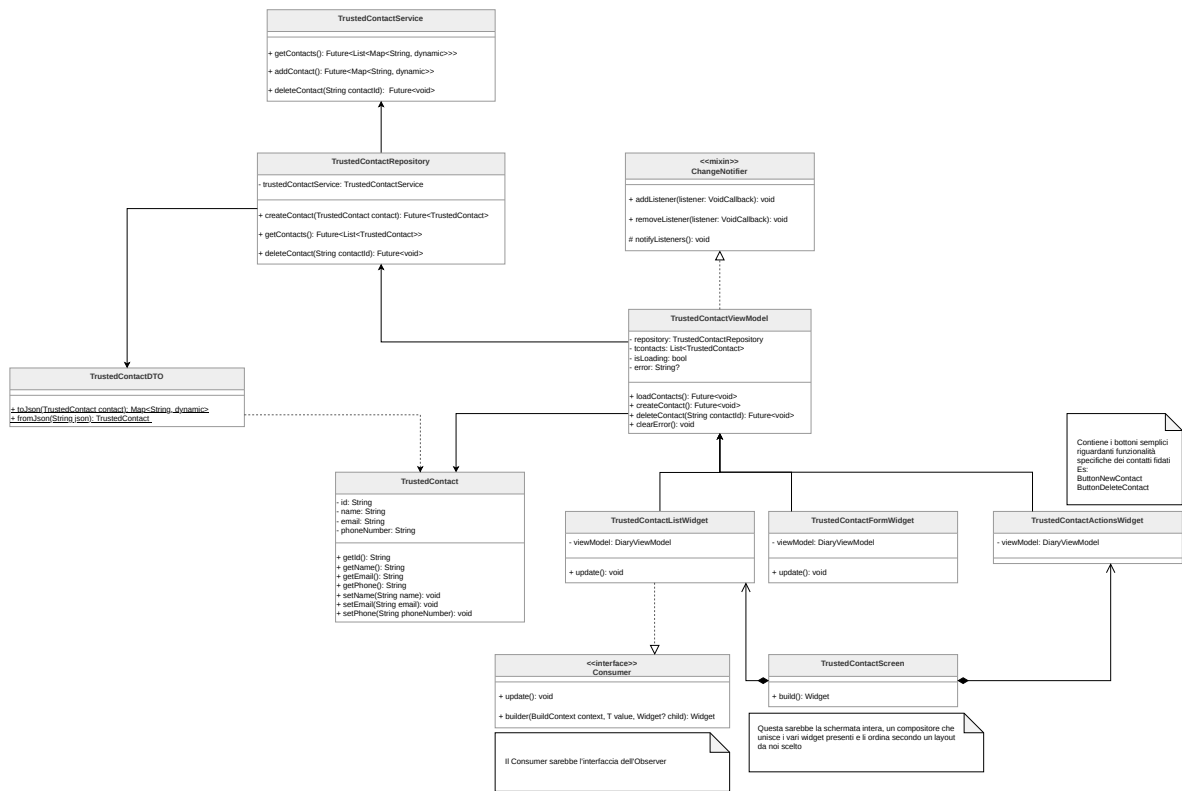


Figura 59: Diagramma delle classi relativo alle funzionalità dei Contatti Fidati



TrustedContactScreen

La classe **TrustedContactScreen** rappresenta la schermata principale dedicata alla gestione dei contatti fidati all'interno del *Presentation Layer*. Essa funge da compositore, unendo i vari widget presenti e ordinandoli secondo il layout stabilito, delegando la logica di stato ai componenti sottostanti.

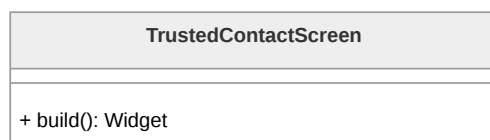


Figura 60: Diagramma della classe **TrustedContactScreen**

TrustedContactListWidget

La classe **TrustedContactListWidget** è responsabile della visualizzazione dell'elenco dei contatti attualmente salvati. Implementando l'interfaccia **Consumer**, osserva il ViewModel per reagire ai cambiamenti di stato e aggiornare dinamicamente l'interfaccia grafica.

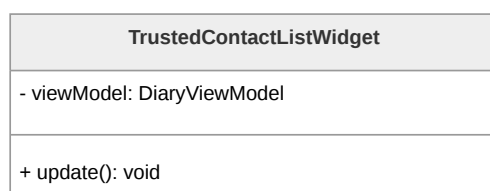


Figura 61: Diagramma della classe **TrustedContactListWidget**

TrustedContactFormWidget

TrustedContactFormWidget gestisce i campi di input necessari per l'inserimento o la modifica dei dati di un contatto fidato. In quanto **Consumer**, si aggiorna in base allo stato del ViewModel.

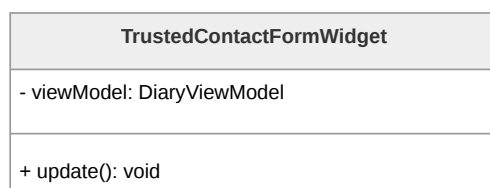


Figura 62: Diagramma della classe **TrustedContactFormWidget**



TrustedContactActionsWidget

Questo widget raggruppa i bottoni e le azioni specifiche per la gestione dei contatti fidati (ad esempio i pulsanti per l'aggiunta di un nuovo contatto o l'eliminazione di uno esistente). Comunica le intenzioni dell'**Utente^G** direttamente al ViewModel.

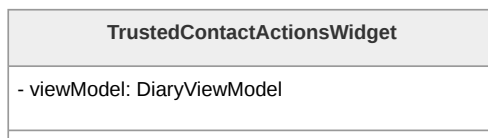


Figura 63: Diagramma della classe **TrustedContactActionsWidget**

TrustedContactViewModel

TrustedContactViewModel è il nucleo della gestione dello stato per questa funzionalità. Mantiene la lista dei contatti (**tcontacts**), lo stato di caricamento (**isLoading**) e la gestione degli errori (**error**). Espone i metodi per caricare, creare e cancellare i contatti, facendo da ponte tra la View e il **Repository^G**. Utilizza il mixin **ChangeNotifier** per notificare i widget in ascolto.

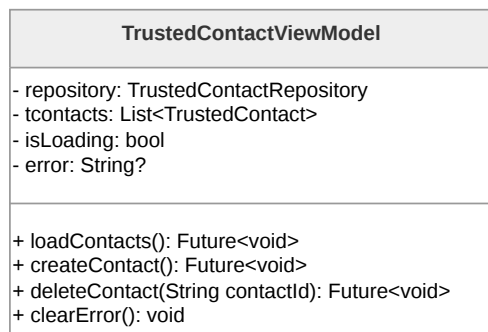


Figura 64: Diagramma della classe **TrustedContactViewModel**

TrustedContact

TrustedContact è la classe di dominio che modella il singolo contatto fidato. Contiene le informazioni anagrafiche e di recapito essenziali, quali l'identificativo unico, il nome, l'indirizzo email e il numero di telefono, fornendo i relativi metodi di accesso e modifica.

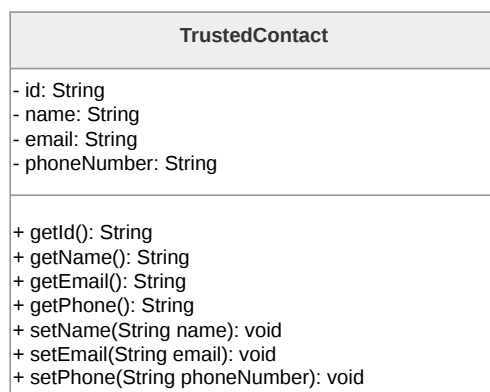


Figura 65: Diagramma della classe **TrustedContact**

TrustedContactRepository

TrustedContactRepository astrae la sorgente dei dati per il ViewModel. Utilizza **TrustedContactService** per eseguire le operazioni CRUD (creazione, lettura, eliminazione), trasformando i dati ricevuti in oggetti di dominio **TrustedContact** e isolando la logica di business dai dettagli di implementazione del network.

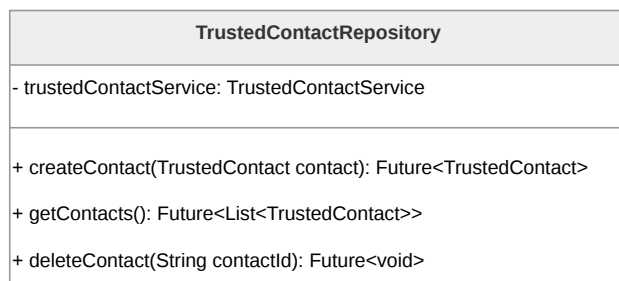


Figura 66: Diagramma della classe **TrustedContactRepository**

TrustedContactService

TrustedContactService appartiene al livello infrastrutturale e si occupa di effettuare fisicamente le chiamate API verso il **Backend^G**. Gestisce le richieste HTTP per ottenere l'elenco dei contatti, aggiungerne di nuovi o eliminarli, restituendo le risposte grezze al **Repository^G**.

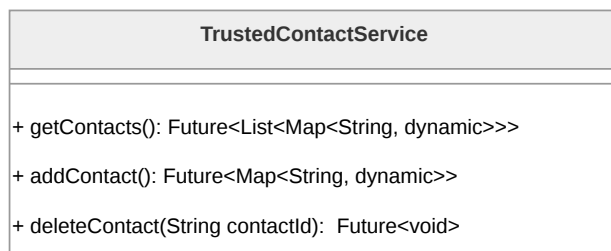


Figura 67: Diagramma della classe **TrustedContactService**

TrustedContactDTO

TrustedContactDTO (*Data Transfer Object*) gestisce la serializzazione e deserializzazione dei dati. Mappa i dati JSON provenienti dall'esterno in oggetti **TrustedContact** e viceversa, garantendo la compatibilità del formato scambiato con le API.

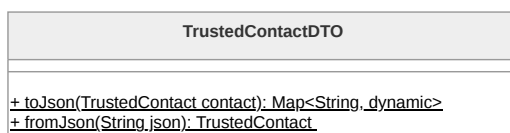


Figura 68: Diagramma della classe **TrustedContactDTO**

3.4.6 Luoghi Sicuri

La funzionalità dei Luoghi Sicuri permette all'**Utente^G** di visualizzare geograficamente, tramite un'interfaccia cartografica, le strutture di supporto, i centri anti-violenza e altri punti di interesse rilevanti. Analogamente al resto dell'applicazione, la funzionalità è implementata secondo il pattern MVVM e le linee guida architetturali di Flutter, distinguendo un **Data Layer^G**, composto da Service e **Repository^G**, e un **UI Layer^G**, composto da View e ViewModel, integrando la libreria ufficiale **google_maps_flutter** per il rendering della mappa.

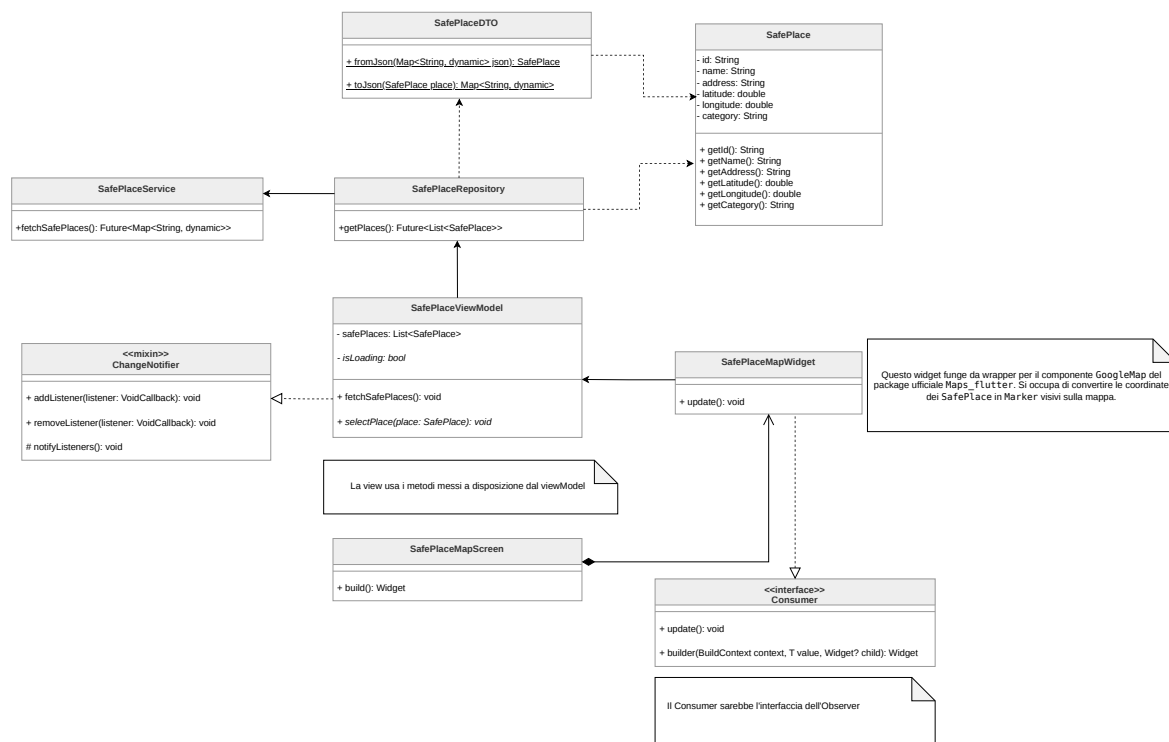


Figura 69: Diagramma delle classi complessivo della funzionalità Luoghi Sicuri



SafePlaceMapScreen La classe **SafePlaceMapScreen** rappresenta la schermata principale dedicata alla visualizzazione dei luoghi sicuri all'interno del *Presentation Layer*. Essa funge da compositore, organizzando il layout della pagina e delegando la logica di visualizzazione cartografica al widget figlio specializzato.

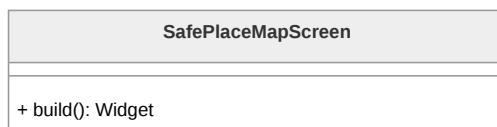


Figura 70: Diagramma della classe **SafePlaceMapScreen**

SafePlaceMapWidget Il componente **SafePlaceMapWidget** incapsula la logica necessaria all'interazione con la cartografia digitale. Implementando l'interfaccia **Consumer**, osserva il ViewModel per reagire ai cambiamenti dei dati. Al suo interno, utilizza il widget **GoogleMap** fornito dal **Framework^G** per visualizzare la mappa e si occupa di mappare gli oggetti di dominio **SafePlace** in entità **Marker** (i "pin" grafici) posizionati in base alle coordinate geografiche caricate.

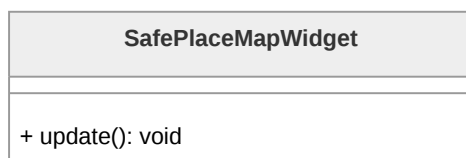


Figura 71: Diagramma della classe **SafePlaceMapWidget**

SafePlaceViewModel **SafePlaceViewModel** rappresenta il nucleo della gestione dello stato per questa funzionalità. Mantiene la lista dei luoghi sicuri (**safePlaces**), gestisce lo stato di caricamento (**isLoading**) e la gestione degli errori. Espone i metodi per il caricamento dei dati dal **Repository^G** e utilizza il mixin **ChangeNotifier** per notificare i widget della View non appena i dati geografici sono pronti per essere visualizzati sulla mappa.

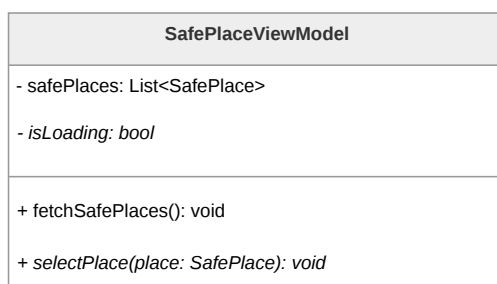


Figura 72: Diagramma della classe **SafePlaceViewModel**



SafePlace **SafePlace** è la classe di dominio che modella il singolo luogo fisico di interesse. Oltre agli attributi identificativi e descrittivi (nome, indirizzo, categoria), include le proprietà fondamentali di latitudine (**latitude**) e longitudine (**longitude**), necessarie per il corretto posizionamento del marker sulla superficie cartografica.

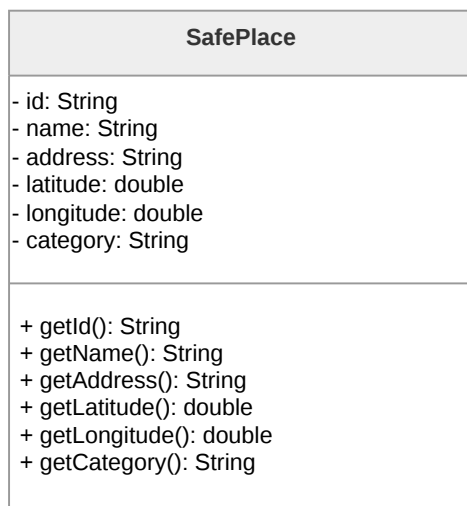


Figura 73: Diagramma della classe **SafePlace**

SafePlaceRepository Il **SafePlaceRepository** astrae la sorgente dei dati geografici per il ViewModel. Utilizza **SafePlaceService** per prelevare le informazioni grezze e le converte in oggetti di dominio **SafePlace** tramite il relativo DTO, garantendo che la logica di business riceva dati tipizzati e pronti all'uso, indipendentemente dalla specifica sorgente dati.

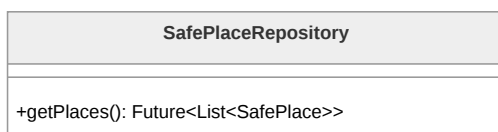


Figura 74: Diagramma della classe **SafePlaceRepository**

SafePlaceService Appartenente al livello infrastrutturale, **SafePlaceService** si occupa di effettuare le chiamate verso il **Backend^G** o sorgenti dati esterne per ottenere l'elenco dei luoghi sicuri in formato grezzo, restituendo le risposte al **Repository^G**.

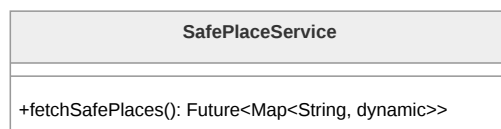


Figura 75: Diagramma della classe **SafePlaceService**

SafePlaceDTO La classe **SafePlaceDTO** (*Data Transfer Object*) gestisce la serializzazione e deserializzazione dei dati geografici. Mappa le chiavi dei campi JSON (incluse le coordinate) in istanze della classe **SafePlace**, assicurando che il formato dei dati scambiati con l'esterno sia compatibile con il modello interno dell'applicazione.

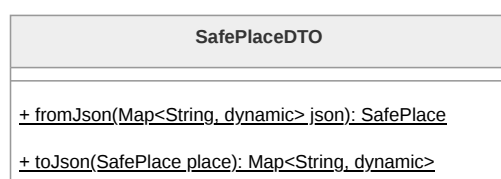


Figura 76: Diagramma della classe **SafePlaceDTO**

3.4.7 Dead Man's Switch

La funzionalità del *Dead Man's Switch* (Interruttore dell'Uomo Morto) rappresenta un sistema di sicurezza critico che invia avvisi automatici in caso di inattività prolungata dell'**Utente**^G. L'**Architettura**^G frontend è focalizzata sulla gestione sicura della configurazione di tale sistema. Il frontend non interagisce direttamente con servizi di posta elettronica, ma demanda la gestione dei timer e l'invio delle email tramite AWS SES al **Backend**^G, garantendo così la sicurezza e l'affidabilità del servizio anche ad applicazione chiusa. Analogamente al resto dell'applicazione, la funzionalità è implementata secondo il pattern MVVM e le linee guida architetturali di Flutter, distinguendo un **Data Layer**^G, composto da Service e **Repository**^G, e un **UI Layer**^G, composto da View e ViewModel.

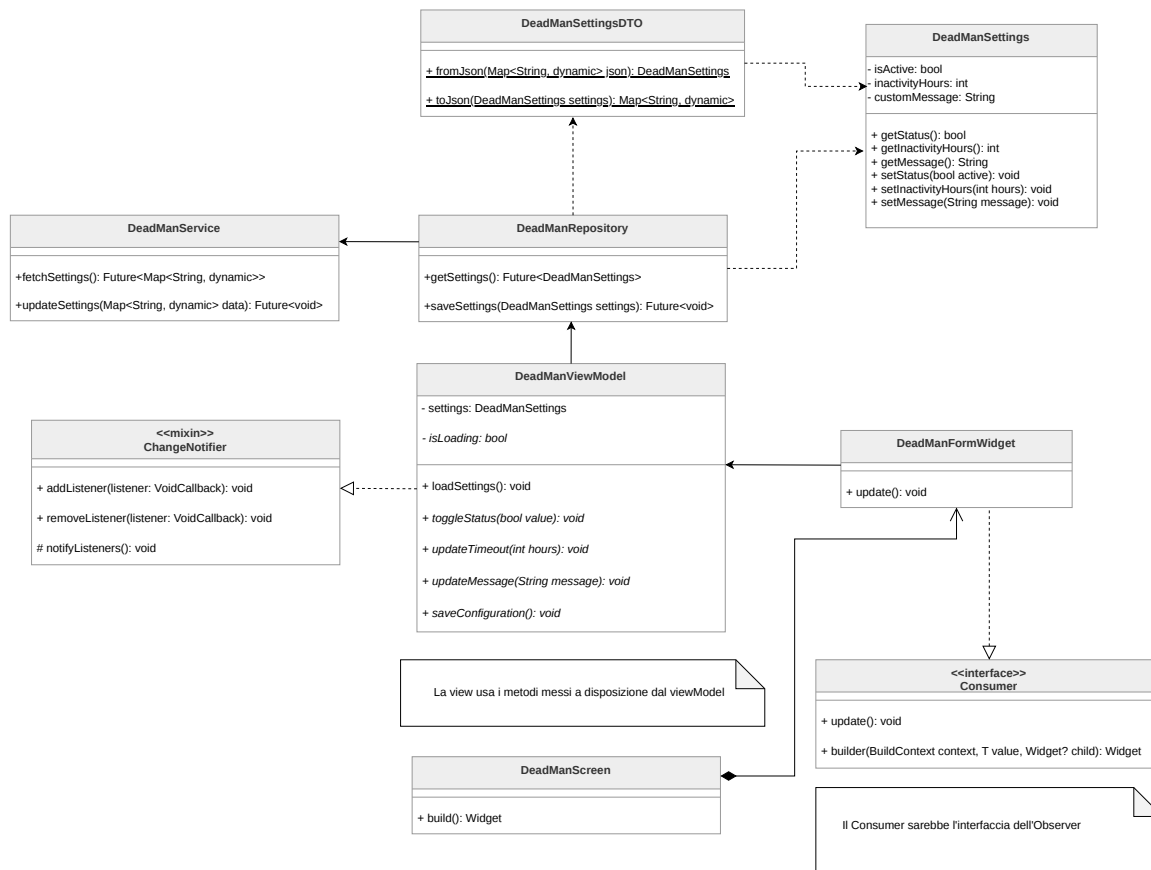


Figura 77: Diagramma delle classi relativo alla funzionalità del Dead Man's Switch



DeadManScreen La classe **DeadManScreen** rappresenta la schermata principale del pannello di controllo per la sicurezza. Funge da compositore, strutturando il layout della pagina e ospitando i form interattivi, delegando al ViewModel la reattività e la **Persistence Logic**^G.

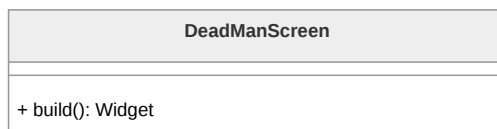


Figura 78: Diagramma della classe **DeadManScreen**

DeadManFormWidget Questo componente incapsula gli elementi interattivi dell'interfaccia, come i selettori (switch) per l'attivazione del servizio, i campi di input per la soglia di inattività (es. ore o giorni) e le aree di testo per il messaggio di emergenza personalizzato. Implementando **Consumer**, aggiorna i propri campi riflettendo fedelmente lo stato mantenuto nel ViewModel.



Figura 79: Diagramma della classe **DeadManFormWidget**

DeadManViewModel **DeadManViewModel** orchestra la logica di business di questa sezione. Mantiene in memoria le configurazioni attuali dell'**Utente**^G (**settings**) e lo stato di sincronizzazione (**isLoading**). Implementa il mixin **ChangeNotifier** per notificare la UI a ogni interazione. Oltre a recuperare i dati, espone metodi fondamentali come **saveConfiguration()**, che permette di confermare e trasmettere al **Repository**^G le nuove impostazioni scelte dall'**Utente**^G.

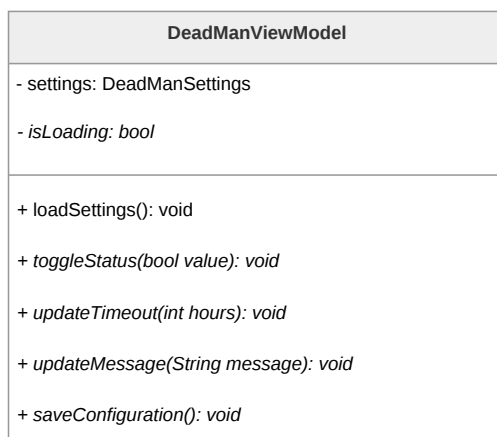


Figura 80: Diagramma della classe **DeadManViewModel**

DeadManSettings **DeadManSettings** è l'entità di dominio che modella le preferenze di sicurezza. Contiene gli attributi necessari a configurare l'allarme: uno stato booleano di attivazione (**isActive**), un valore temporale che definisce la soglia di tolleranza (**inactivityHours**) e il corpo del messaggio testuale (**customMessage**) che il **Backend^G** inoltrerà ai contatti fidati tramite AWS SES.

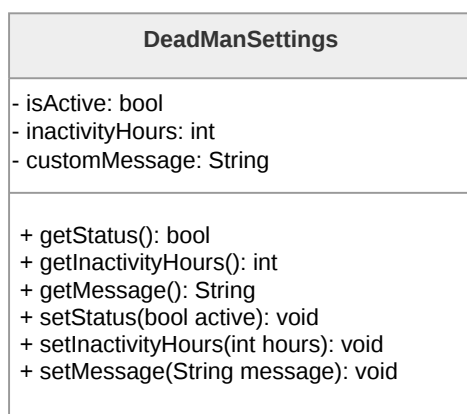


Figura 81: Diagramma della classe **DeadManSettings**

DeadManRepository Il **DeadManRepository** gestisce la persistenza e il recupero delle configurazioni. Agisce da ponte tra la **Presentation Logic^G** e il livello infrastrutturale, utilizzando **DeadManService** per inviare le preferenze aggiornate e convertendo le risposte di rete in oggetti di dominio **DeadManSettings** sicuri e validati tramite il DTO.

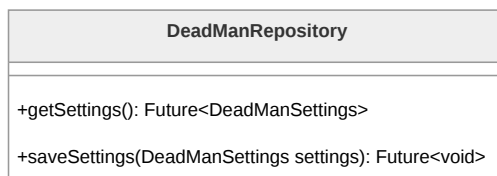


Figura 82: Diagramma della classe **DeadManRepository**

DeadManService Appartenente al **Data Layer^G**, **DeadManService** si occupa della comunicazione HTTP/REST con le API del **Backend^G**. Dispone dei metodi necessari per effettuare le operazioni di *GET* (lettura parametri) e *PUT/POST* (aggiornamento configurazioni) sul database remoto.

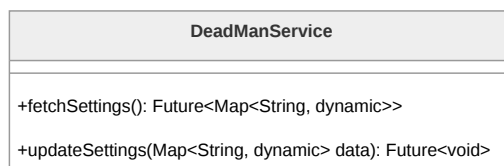


Figura 83: Diagramma della classe **DeadManService**

DeadManSettingsDTO Il **DeadManSettingsDTO** è l'oggetto di trasferimento dati incaricato della serializzazione. Consente di mappare i campi JSON inviati e ricevuti dal server, assicurando che la comunicazione tra l'app Flutter e il **Backend^G** mantenga un **Contratto^G** dati rigoroso.

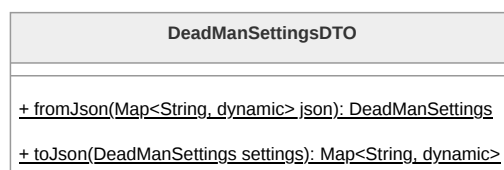


Figura 84: Diagramma della classe **DeadManSettingsDTO**

3.4.8 Area Informativa

L'Area Informativa costituisce un centro di risorse in sola lettura per l'**Utente^G**, aggregando informazioni su community di supporto, riferimenti normativi e materiale divulgativo. La funzionalità è implementata secondo il pattern MVVM e le linee guida architetturali di Flutter, distinguendo un **Data Layer^G**, composto da Service e **Repository^G**, e un **UI Layer^G**, composto da View e ViewModel.

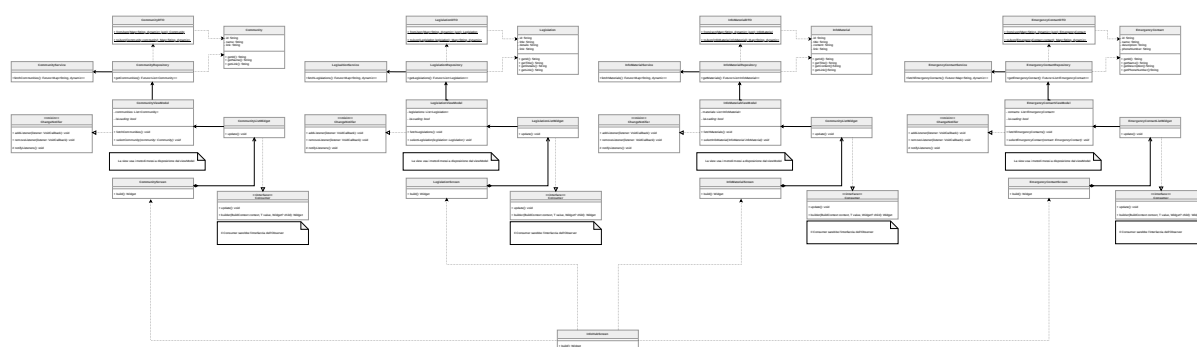


Figura 85: Diagramma delle classi complessivo per l'Area Informativa



InfoHubScreen La classe **InfoHubScreen** funge da punto di ingresso per l'intera area informativa. Come componente del *Presentation Layer*, organizza il layout principale e gestisce la navigazione verso le tre sottosezioni specifiche: Community, Legislazioni e Materiale Informativo. Essendo una classe View e quindi solamente adibita alla visualizzazione di widget, non contiene particolare logica interna.

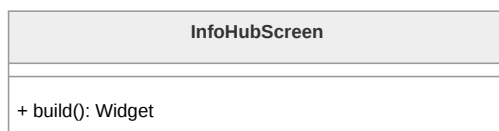


Figura 86: Diagramma della classe **InfoHubScreen**

CommunityScreen **CommunityScreen** rappresenta la vista principale per la funzionalità di consultazione delle community. Organizza la struttura e l'ordinamento dei widget per la lista e il dettaglio, delegando la *Presentation Logic*^G e il recupero dei dati al relativo ViewModel.

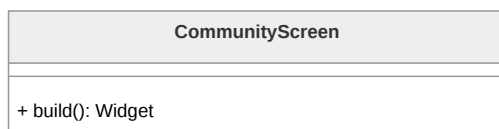


Figura 87: Diagramma della classe **CommunityScreen**

CommunityListWidget Responsabile della renderizzazione dell'elenco delle community, **CommunityListWidget** implementa l'interfaccia **Consumer**. Questo gli permette di osservare il **CommunityViewModel** e aggiornare la visualizzazione in seguito ad una notifica dal ViewModel.

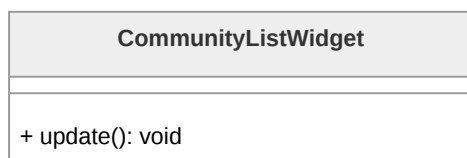


Figura 88: Diagramma della classe **CommunityListWidget**

CommunityViewModel **CommunityViewModel** è il nucleo logico della sezione. Gestisce la lista di oggetti **Community**, lo stato di caricamento (**isLoading**) per la gestione dei feedback visivi (spinner) e i messaggi di errore. Utilizza il mixin **ChangeNotifier** per notificare la View in seguito ad un cambiamento di stato dei dati.

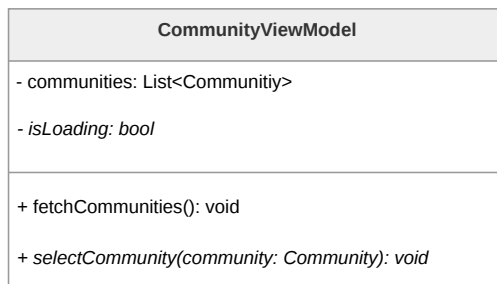


Figura 89: Diagramma della classe **CommunityViewModel**

Community Classe di dominio che modella le informazioni di una community di supporto. Include attributi fondamentali quali l'identificativo, il nome, la descrizione e l'URL di riferimento, fornendo i metodi necessari per l'accesso ai dati in sola lettura.

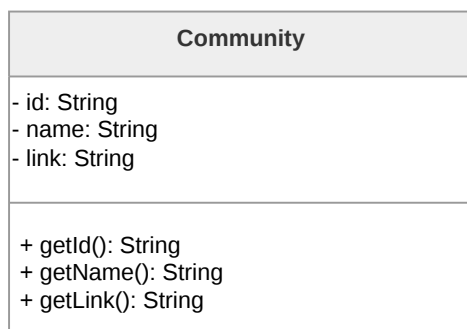


Figura 90: Diagramma della classe **Community**

CommunityRepository La classe **CommunityRepository** funge da intermediario tra il ViewModel e il Service. Si occupa di richiedere i dati grezzi, gestirne la conversione tramite il DTO e restituire oggetti di dominio **Community** "puliti", isolando la **Business Logic**^G dai dettagli di implementazione della rete.

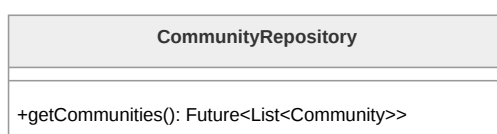


Figura 91: Diagramma della classe **CommunityRepository**

CommunityService Appartenente al **Data Layer**^G, **CommunityService** esegue le richieste HTTP verso le API del **Backend**^G per ottenere l'elenco delle community salvate nel database.



Figura 92: Diagramma della classe **CommunityService**

CommunityDTO La classe **CommunityDTO** gestisce la logica di mappatura dei dati. Implementa i metodi **fromJson** e **toJson** per trasformare le risposte JSON provenienti dal network in istanze della classe **Community**.



Figura 93: Diagramma della classe **CommunityDTO**

LegislationScreen **LegislationScreen** rappresenta la vista principale per la funzionalità di consultazione delle leggi. Organizza la struttura e l'ordinamento dei widget per la lista e il dettaglio, delegando la **Presentation Logic**^G e il recupero dei dati al relativo ViewModel.

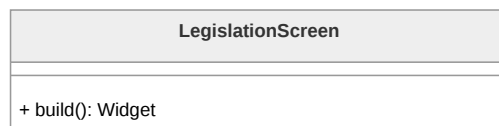


Figura 94: Diagramma della classe **LegislationScreen**

LegislationListWidget Responsabile della renderizzazione dell'elenco delle leggi, **LegislationListWidget** implementa l'interfaccia **Consumer**. Questo gli permette di osservare il **LegislationViewModel** e aggiornare la visualizzazione in seguito ad una notifica dal ViewModel.



Figura 95: Diagramma della classe **LegislationListWidget**



LegislationViewModel **LegislationViewModel** è il nucleo logico della sezione. Gestisce la lista di oggetti **Legislation**, lo stato di caricamento (**isLoading**) per la gestione dei feedback visivi (spinner) e i messaggi di errore. Utilizza il mixin **ChangeNotifier** per notificare la View in seguito ad un cambiamento di stato dei dati.

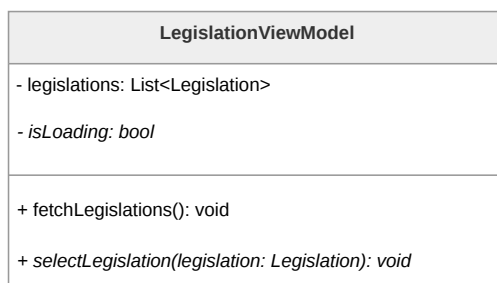


Figura 96: Diagramma della classe **LegislationViewModel**

Legislation Classe di dominio che modella le informazioni di una legge o di un riferimento normativo. Include attributi fondamentali quali l'identificativo, il titolo, il testo o la descrizione dell'articolo e l'URL di riferimento alla fonte ufficiale, fornendo i metodi necessari per l'accesso ai dati in sola lettura.

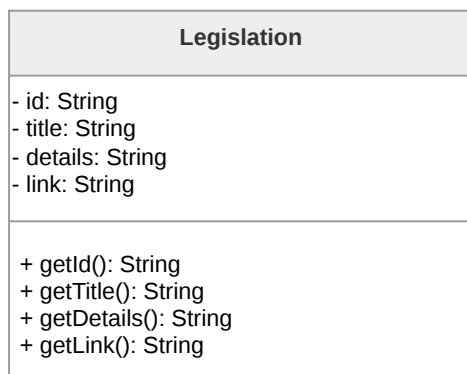


Figura 97: Diagramma della classe **Legislation**

LegislationRepository La classe **LegislationRepository** funge da intermediario tra il ViewModel e il Service. Si occupa di richiedere i dati grezzi, gestirne la conversione tramite il DTO e restituire oggetti di dominio **Legislation** "puliti", isolando la **Business Logic**^G dai dettagli di implementazione della rete.



Figura 98: Diagramma della classe **LegislationRepository**

LegislationService Appartenente al **Data Layer^G**, **LegislationService** esegue le richieste HTTP verso le API del **Backend^G** per ottenere l'elenco delle leggi salvate nel database.



Figura 99: Diagramma della classe **LegislationService**

LegislationDTO La classe **LegislationDTO** gestisce la logica di mappatura dei dati. Implementa i metodi **fromJson** e **toJson** per trasformare le risposte JSON provenienti dal network in istanze della classe **Legislation**.

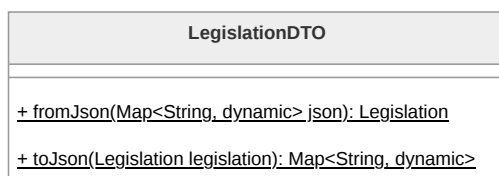


Figura 100: Diagramma della classe **LegislationDTO**

InfoMaterialScreen **InfoMaterialScreen** rappresenta la vista principale per la funzionalità di consultazione di materiale informativo e divulgativo. Organizza la struttura e l'ordinamento dei widget per la lista e il dettaglio, delegando la **Presentation Logic^G** e il recupero dei dati al relativo ViewModel.

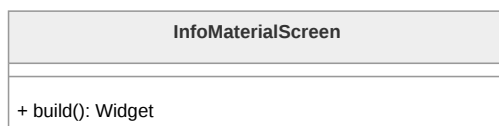


Figura 101: Diagramma della classe **InfoMaterialScreen**

InfoMaterialListWidget Responsabile della renderizzazione dell'elenco del materiale informativo, **InfoMaterialListWidget** implementa l'interfaccia **Consumer**. Questo gli



permette di osservare il **InfoMaterialViewModel** e aggiornare la visualizzazione in seguito ad una notifica dal ViewModel.

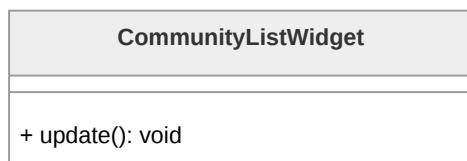


Figura 102: Diagramma della classe **InfoMaterialListWidget**

InfoMaterialViewModel **InfoMaterialViewModel** è il nucleo logico della sezione. Gestisce la lista di oggetti **InfoMaterial**, lo stato di caricamento (**isLoading**) per la gestione dei feedback visivi (spinner) e i messaggi di errore. Utilizza il mixin **ChangeNotifier** per notificare la View in seguito ad un cambiamento di stato dei dati.

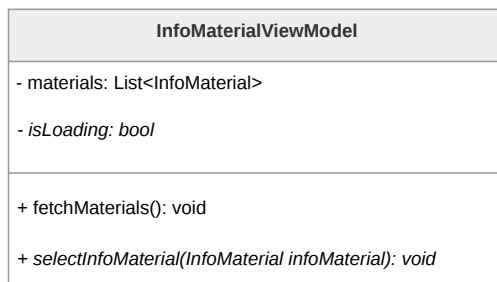


Figura 103: Diagramma della classe **InfoMaterialViewModel**

InfoMaterial Classe di dominio che modella le informazioni di un materiale informativo. Include attributi fondamentali quali l'identificativo, il titolo, il contenuto e eventuale URL di riferimento, fornendo i metodi necessari per l'accesso ai dati in sola lettura.

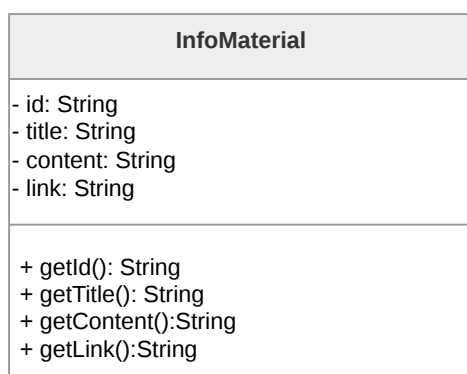


Figura 104: Diagramma della classe **InfoMaterial**



InfoMaterialRepository La classe **InfoMaterialRepository** funge da intermediario tra il ViewModel e il Service. Si occupa di richiedere i dati grezzi, gestirne la conversione tramite il DTO e restituire oggetti di dominio **InfoMaterial** "puliti", isolando la **Business Logic^G** dai dettagli di implementazione della rete.

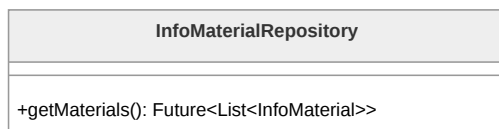


Figura 105: Diagramma della classe **InfoMaterialRepository**

InfoMaterialService Appartenente al **Data Layer^G**, **InfoMaterialService** esegue le richieste HTTP verso le API del **Backend^G** per ottenere l'elenco dei materiali informativi salvati nel database.

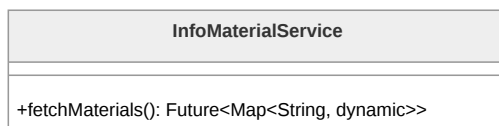


Figura 106: Diagramma della classe **InfoMaterialService**

InfoMaterialDTO La classe **InfoMaterialDTO** gestisce la logica di mappatura dei dati. Implementa i metodi **fromJson** e **toJson** per trasformare le risposte JSON provenienti dal network in istanze della classe **InfoMaterial**.

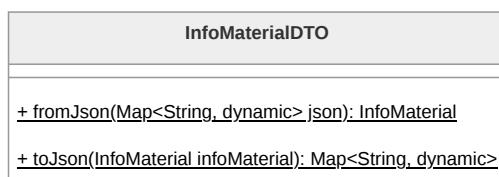


Figura 107: Diagramma della classe **InfoMaterialDTO**

EmergencyContactScreen **EmergencyContactScreen** rappresenta la vista principale per la consultazione rapida dei contatti di emergenza o degli enti di supporto preposti. Organizza la struttura e l'ordinamento dei widget, delegando la **Presentation Logic^G** e il recupero dei dati al relativo ViewModel.

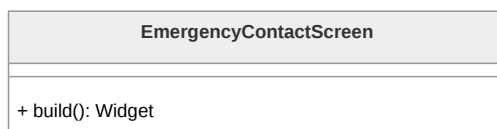


Figura 108: Diagramma della classe **EmergencyContactScreen**

EmergencyContactListWidget Responsabile della renderizzazione dell'elenco dei contatti, **EmergencyContactListWidget** implementa l'interfaccia **Consumer**. Questo gli permette di osservare il **EmergencyContactViewModel** e aggiornare la visualizzazione in seguito ad una notifica dal ViewModel. Inoltre, gestisce l'interazione dell'**Utente**^G: alla selezione di una voce, intercetta l'evento e segnala l'intenzione di avviare una chiamata telefonica.

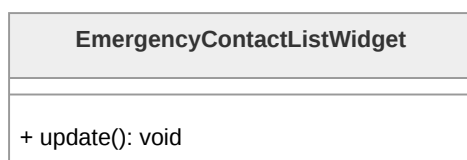


Figura 109: Diagramma della classe **EmergencyContactListWidget**

EmergencyContactViewModel **EmergencyContactViewModel** è il nucleo logico della sezione. Gestisce la lista di oggetti **EmergencyContact** e lo stato di caricamento (**isLoading**). Utilizza il mixin **ChangeNotifier** per notificare la View in seguito ad un cambiamento di stato dei dati. Inoltre, espone metodi per gestire l'avvio della telefonata (ad esempio interfacciandosi con servizi di sistema tramite *intent* o URL launcher) quando un contatto viene selezionato.

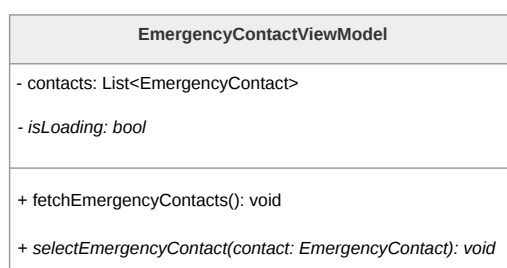


Figura 110: Diagramma della classe **EmergencyContactViewModel**

EmergencyContact Classe di dominio che modella le informazioni di un ente o contatto di emergenza. Include attributi fondamentali e immediati quali l'identificativo, il nome dell'ente e il numero di telefono (**phoneNumber**), fornendo i metodi necessari per l'accesso ai dati in sola lettura.

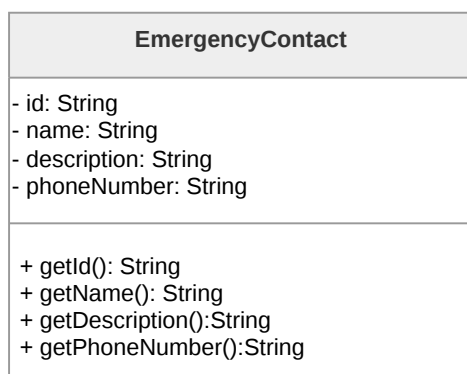


Figura 111: Diagramma della classe **EmergencyContact**

EmergencyContactRepository La classe **EmergencyContactRepository** funge da intermediario tra il ViewModel e il Service. Si occupa di richiedere i dati grezzi, gestirne la conversione tramite il DTO e restituire oggetti di dominio **EmergencyContact** "puliti", isolando la **Business Logic^G** dai dettagli di implementazione della rete.



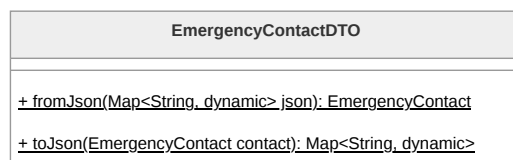
Figura 112: Diagramma della classe **EmergencyContactRepository**

EmergencyContactService Appartenente al **Data Layer^G**, **EmergencyContactService** esegue le richieste verso il **Backend^G** per ottenere l'elenco aggiornato dei contatti di emergenza salvati nel database.



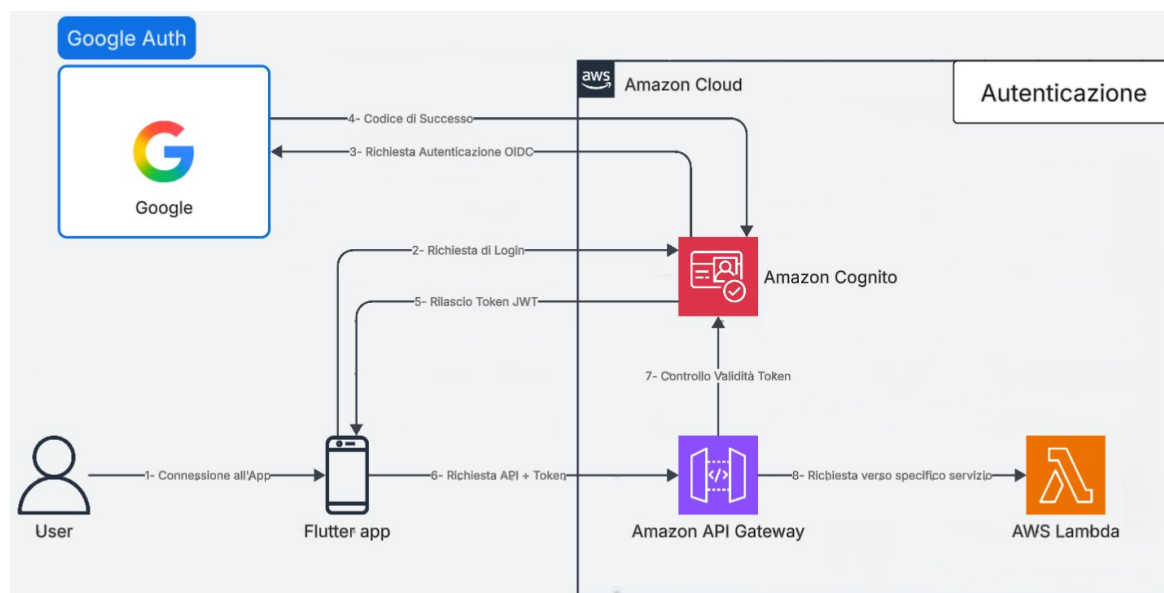
Figura 113: Diagramma della classe **EmergencyContactService**

EmergencyContactDTO La classe **EmergencyContactDTO** gestisce la logica di mappatura dei dati. Implementa i metodi di conversione JSON per trasformare i **Payload^G** di rete in istanze della classe **EmergencyContact**.

Figura 114: Diagramma della classe **EmergencyContactDTO**

3.5. Architettura Back End

3.5.1 Autenticazione

Figura 115: **Architettura^G** e Flusso di Autenticazione

Architettura^G e Flusso di Autenticazione: Modulo Federated Login

La presente sezione descrive l'**Architettura^G** logica e il flusso dei dati relativi al modulo di autenticazione e autorizzazione dell'applicazione. Il sistema adotta un modello di federated login basato su Amazon Cognito integrato con Google come Identity Provider (IdP), progettato per garantire un accesso sicuro e standardizzato alle risorse di **Backend^G** senza demandare la gestione crittografica delle credenziali all'applicazione client.



Iniziazione del Flusso e Autenticazione Federata

Il **Processo^G** di autenticazione ha inizio quando l'**Utente^G** interagisce con l'applicazione client (Flutter) richiedendo l'accesso al sistema. L'applicazione delega interamente la gestione dell'identità inoltrando una richiesta di login ad Amazon Cognito. Cognito, fungendo da Service Provider, intercetta la richiesta e avvia una procedura di autenticazione basata sullo standard OpenID Connect (OIDC) delegandola ai server di Google. L'**Utente^G** completa la fase di riconoscimento interfacciandosi direttamente con l'infrastruttura del provider esterno. A seguito della corretta **Verifica^G** delle credenziali, Google restituisce ad Amazon Cognito un codice di autorizzazione attestante il successo dell'operazione e l'identità dell'**Utente^G**.

Emissione dei Token di Sessione

Acquisita la conferma dal provider federato, Amazon Cognito processa il codice di autorizzazione e genera un set di token crittografici aderenti allo standard JWT (JSON Web Token). Questi token vengono trasmessi in modo sicuro all'applicazione mobile, concludendo la fase di autenticazione. Da questo momento, il client dispone di una prova di identità verificata e utilizzerà l'Access Token JWT per autorizzare le successive chiamate di rete dirette ai servizi di **Backend^G**.

Validazione^G Perimetrale e Instradamento Sicuro

Per accedere alle logiche applicative, l'app Flutter compone una richiesta HTTPS indirizzata all'Amazon API Gateway, includendo il token JWT nelle intestazioni di autorizzazione. L'API Gateway assume il ruolo di strato di sicurezza perimetrale (edge security). Prima di instradare il traffico, il Gateway utilizza un Cognito Authorizer nativo per eseguire un rigoroso controllo di validità sul token ricevuto. Tale procedura **Verifica^G** l'autenticità della firma crittografica, l'integrità del **Payload^G** e la validità temporale della sessione direttamente contro le configurazioni dello User Pool di Cognito.

Questa specifica impostazione architetturale assicura che qualsiasi richiesta malevola, non autorizzata o provvista di credenziali scadute venga respinta al livello di rete. Solamente le richieste che superano con successo la **Validazione^G** del token vengono instradate dall'API Gateway verso la specifica funzione AWS Lambda designata, garantendo che le risorse computazionali elaborino esclusivamente traffico certificato e sicuro.



3.5.2 Luoghi sicuri

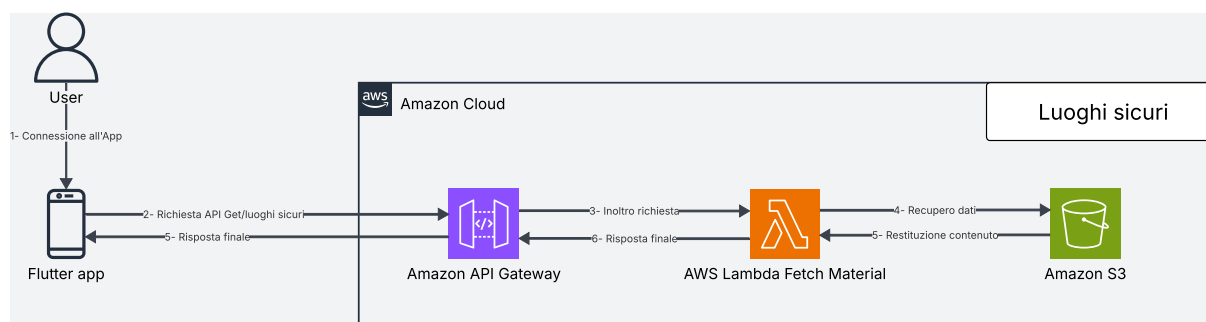


Figura 116: **Architettura^G** moduli Luoghi Sicuri

Architettura^G e Flusso di Elaborazione: Modulo Luoghi Sicuri

Il modulo "Luoghi Sicuri" è stato ingegnerizzato adottando una rigorosa separazione delle responsabilità (*Separation of Concerns^G*) tra l'elaborazione lato client (Frontend) e l'infrastruttura **Cloud^G** (**Backend^G**). L'obiettivo primario è stato quello di minimizzare il carico computazionale server-side ed eliminare del tutto i costi legati ai calcoli geospaziali.

Il flusso di elaborazione si articola nei seguenti passaggi:

- **Ingestione della Richiesta:** L'**Utente^G** accede alla sezione della mappa tramite l'applicazione. Il client Flutter genera una richiesta HTTPS (metodo `GET /safe_locations`).
- **Instradamento verso Lambda:** API Gateway intercetta la richiesta, ne valida il contesto e la inoltra alla funzione AWS Lambda dedicata alla gestione dei luoghi sicuri.
- **Accesso ai dati e logica applicativa:** la Lambda esegue la logica applicativa necessaria e recupera i dati dal bucket S3 dedicato. Al suo interno sono infatti memorizzati sia i metadati descrittivi che coordinate geospaziali (latitudine e longitudine) per ciascun punto di interesse.
- **Elaborazione e serializzazione:** la funzione Lambda elabora i dati estratti e li converte in un payload JSON standardizzato e ottimizzato per il consumo lato client.
- **Risposta al client:** API Gateway inoltra la risposta HTTP 200 OK al client Flutter contenente l'elenco dei luoghi sicuri.



Delega della Logica Geospaziale al Livello Client (Edge Computing)

Al fine di ottimizzare ulteriormente i costi, l'intera logica di rendering visivo e di routing è delegata all'applicazione sul dispositivo dell'**Utente^G**. Il **Backend^G** AWS si configura esclusivamente come Data Provider di coordinate statiche. Alla ricezione del **Payload^G** JSON, l'applicativo Flutter si fa carico di istanziare la mappa nativa e posizionare i marker grafici (pin). L'azione di navigazione guidata (es. pulsante "Portami lì") sfrutta le Intents (su Android) o le URL Schemes (su iOS) del sistema operativo per invocare applicazioni di navigazione di terze parti (come Google Maps o Apple Maps). Questo approccio demanda il tracciamento GPS e il calcolo dinamico dell'itinerario al dispositivo locale, garantendo la scalabilità infinita del sistema AWS anche in condizioni di massiccio utilizzo concorrente.

API associate

- **GET /safe_locations/**: ritorna le informazioni di tutti i luoghi sicuri.

Nota di sicurezza: L'integrazione di un Authorizer Cognito per la **Verifica^G** delle autorizzazioni è stata deliberatamente omessa per questa rotta. L'assenza dell'Authorizer in quest'area è difatti una scelta architetturale mirata a rimuovere complessità infrastrutturale non necessaria e ad abbattere i costi operativi associati al recupero di questi specifici dati.

3.5.3 Materiale informativo

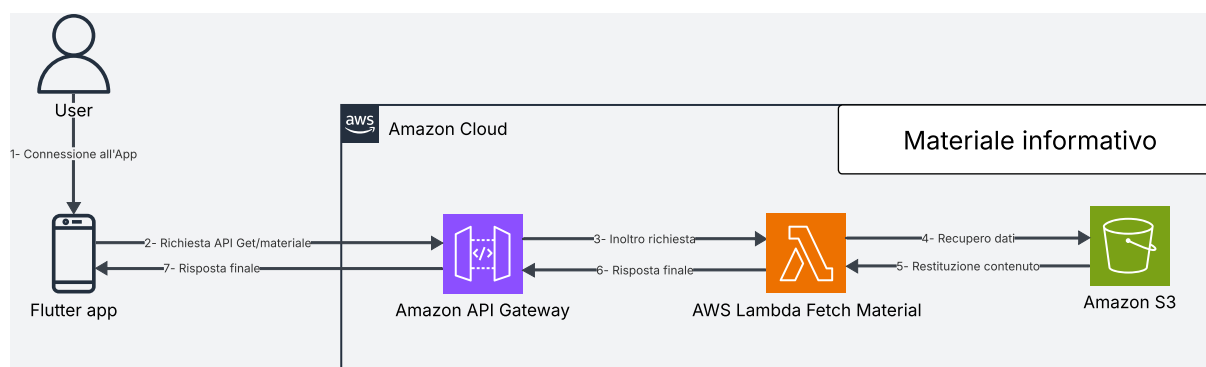


Figura 117: **Architettura^G** moduli Materiale Informativo

Architettura^G e Flusso di Elaborazione: Modulo Materiale Informativo

La sezione dedicata al "Materiale Informativo" è stata progettata privilegiando la massima efficienza e l'ottimizzazione dei costi operativi. Trattandosi di un modulo a prevalenza di lettura (Read-Heavy) e privo di logiche di business complesse per la manipolazione dei



dati, l'**Architettura^G** prevede una funzione Lambda molto leggera, che si interfaccia con S3 esclusivamente per il fetch dei dati.

Il ciclo di elaborazione si innesca quando l'applicazione client necessita di recuperare il catalogo delle informazioni, generando una richiesta HTTPS (metodo **GET**) verso l'endpoint dedicato. L'API Gateway intercetta la chiamata e la inoltra alla funzione AWS Lambda responsabile del modulo.

La Lambda si occupa quindi di:

- recuperare i record relativi ai materiali informativi dal bucket S3;
- applicare eventuali trasformazioni o filtraggi dei dati;
- serializzare il risultato in un formato JSON standard, leggero e consumabile dal frontend.

Il risultato elaborato viene restituito all'API Gateway, che lo inoltra al client Flutter tramite risposta HTTP. Il ciclo sincrono si conclude con la consegna del catalogo informativo all'applicazione.

API associate

- **GET /learning/**: ritorna le *preview* di tutti i materiali di apprendimento.
- **GET /learning/{learning_id}**: ritorna il contenuto di uno specifico materiale di apprendimento.

Nota di sicurezza: In maniera analoga a quanto previsto per i luoghi sicuri, anche per l'accesso al materiale informativo si è optato per non implementare l'Authorizer Cognito. Questa decisione è dettata dalla volontà di snellire l'**Architettura^G**, riducendo le complessità superflue ed evitando di incorrere nei costi operativi aggiuntivi dell'infrastruttura di autenticazione.



3.5.4 Diario criptato e fittizio

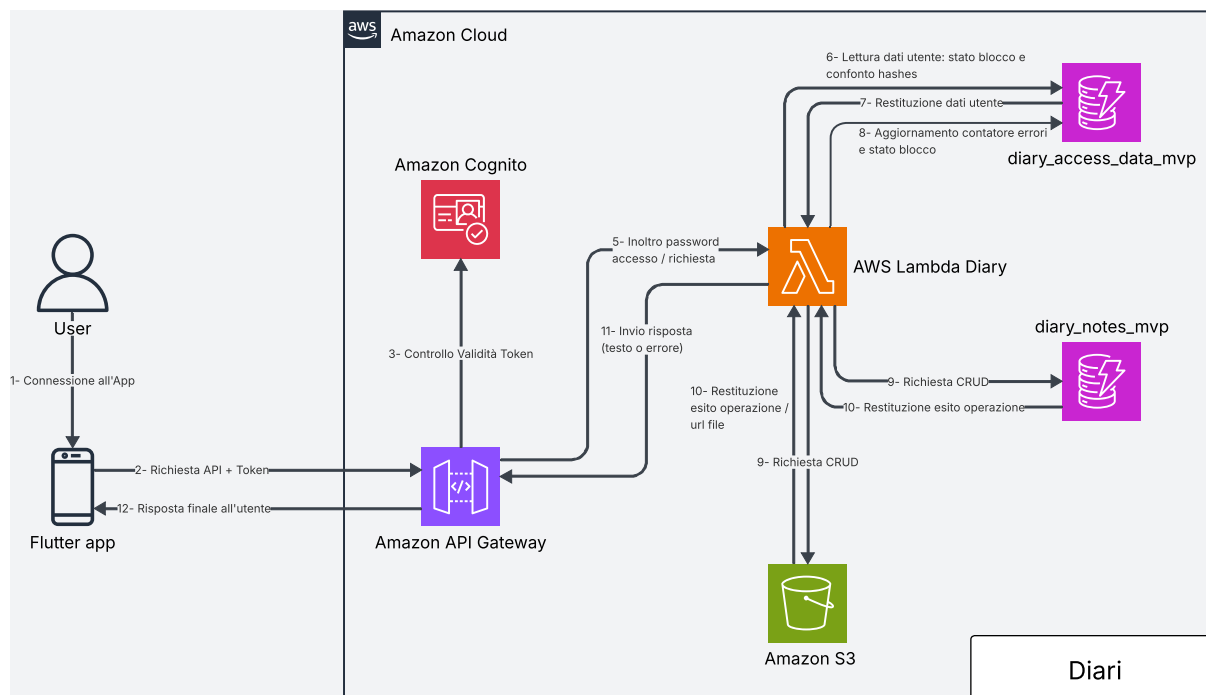


Figura 118: **Architettura^G** diari

Architettura^G e Flusso di Elaborazione: diario criptato e fittizio

La sezione dedicata al diario criptato e fittizio è progettata per garantire la sicurezza delle informazioni dell'utente e la corretta gestione e archiviazione di diverse tipologie di dati, quali contenuti testuali, immagini e tracce audio. A differenza delle altre funzionalità, il diario è l'unica componente protetta da una password aggiuntiva, distinta da quella utilizzata per l'account principale.

Questo approccio consente all'applicazione di fornire accesso a risorse differenti in base alla password inserita, dato che i due diari sono protetti da credenziali separate. Poiché Amazon Cognito non supporta direttamente flussi di autenticazione di questo tipo, la gestione dell'accesso al diario avviene tramite la memorizzazione in DynamoDB degli hash delle due password. Quando l'utente tenta l'accesso, viene generato l'hash della password inserita tramite Argon2id. Questo hash viene quindi confrontato con quello salvato nella tabella utenti. In caso di corrispondenza, la password fornita viene utilizzata per derivare una chiave crittografica, impiegata per decifrare le note del diario corrispondente. In entrambi i casi viene utilizzato Argon2id, ma con parametri di memory cost, time cost, salt length e parallelism diversi.

Una volta completato con successo l'accesso, viene creata una sessione lato backend, così da rendere più efficienti le successive operazioni CRUD. La creazione della sessione consiste nella generazione di un token casuale che viene inviato in risposta al client. Tale token ha



durata limitata e viene memorizzato nella tabella DynamoDB relativa all'accesso al diario. In caso di un numero di tentativi di accesso errati superiore alla soglia consentita, viene applicato un meccanismo di timeout incrementale che blocca temporaneamente ulteriori tentativi.

L'archiviazione delle note avviene su due servizi AWS distinti. I contenuti testuali sono memorizzati in DynamoDB, che mantiene le informazioni per ciascun utente e per ciascun diario. I file binari, come immagini e tracce audio, sono invece salvati in bucket S3. Il recupero di tali risorse avviene tramite URL presigned, al fine di ridurre la quantità di dati trasferiti sulla rete.

Il flusso di elaborazione è dunque il seguente:

- **Invio della Richiesta di Autenticazione:** L'Utente inserisce la password del diario tramite app. Il client genera una richiesta HTTPS (metodo POST /diary/auth/) includendo la password nel body della richiesta.
- **Verifica delle Credenziali:** API Gateway verifica che l'utente sia autenticato all'applicazione. Dopodiché inoltra la richiesta alla funzione Lambda, che calcola l'hash della password ricevuta e lo confronta con gli hash memorizzati in DynamoDB. In caso di corrispondenza, viene identificato il tipo di diario associato.
- **Derivazione della Chiave Crittografica:** La funzione Lambda utilizza la password fornita per derivare una chiave crittografica tramite Argon2id, impiegando un salt dedicato. Tale chiave viene utilizzata per le successive operazioni di decrittazione.
- **Creazione della Sessione:** A seguito dell'autenticazione, il backend genera una sessione associata all'utente, che consente di effettuare operazioni sul diario senza ripetere il processo di autenticazione per ogni richiesta.
- **Recupero delle Note:** Quando l'Utente richiede di recuperare la lista di note di uno dei diari (metodo GET /diary/{diary_type}/), API Gateway invia la richiesta alla Lambda, che recupera le *preview* dal database DynamoDB e le restituisce al client.
- **Recupero di una Nota:** Per ottenere il contenuto completo di una nota (metodo GET /diary/{diary_type}/{note_id}/), la Lambda recupera i dati cifrati da DynamoDB e gli eventuali riferimenti a file su S3, decrittando il contenuto tramite la chiave derivata e restituendolo al client.
- **Gestione dei Contenuti Binari:** In presenza di immagini o tracce audio, la Lambda genera URL presigned per gli oggetti memorizzati su S3, permettendo al client di accedere direttamente alle risorse senza transitare dal backend.



- **Operazioni di Scrittura:** Per la creazione o modifica di note (metodi POST /diary/{diary_type}/ e PUT /diary/{diary_type}/{note_id}/), la Lambda cifra i contenuti tramite la chiave derivata e li memorizza in DynamoDB, salvando eventuali file binari su S3.
- **Eliminazione delle Note:** In caso di richiesta di eliminazione (metodo DELETE /diary/{diary_type}/{note_id}/), la Lambda rimuove i dati associati dal database e, se presenti, elimina anche i file binari corrispondenti dai bucket S3.

API associate

- POST /diary/auth/: invia la password per l'autenticazione; ritorna l'esito dell'autenticazione e il tipo di diario associato.
- GET /diary/{diary_type}/: ritorna le *preview* di tutte le note del diario specificato.
- POST /diary/{diary_type}/: crea una nuova nota per l'utente nel diario specificato.
- GET /diary/{diary_type}/{note_id}/: ritorna il contenuto completo della nota specificata.
- PUT /diary/{diary_type}/{note_id}/: aggiorna la nota specificata nel database.
- DELETE /diary/{diary_type}/{note_id}/: rimuove la nota specificata.

3.5.5 Chatbot

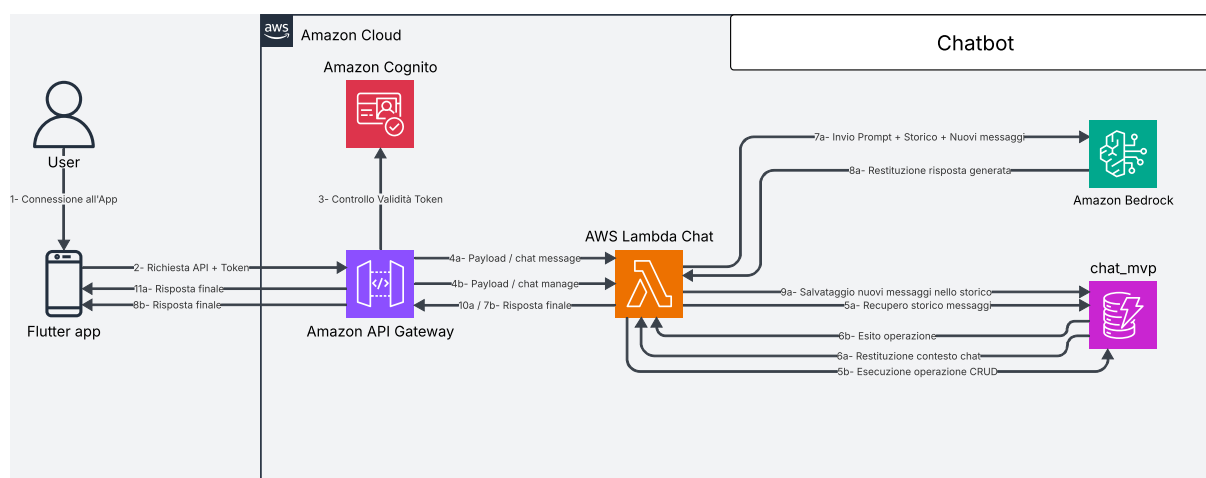


Figura 119: Architettura^G chatbot



Architettura^G e Flusso di Elaborazione: chatbot (detective delle relazioni e specchio intelligente)

L'architettura di backend dedicata al chatbot è progettata per gestire conversazioni persistenti tra utente e modello linguistico, garantendo scalabilità, efficienza nell'accesso ai dati e qualità delle risposte generate.

API Gateway rappresenta il punto di ingresso per tutte le richieste client e si occupa della verifica dell'autenticazione dell'utente. Una volta validata la richiesta, questa viene inoltrata a una funzione Lambda che implementa la logica applicativa del chatbot, inclusa la gestione delle chat e l'interazione con il modello linguistico.

Le conversazioni sono strutturate come entità persistenti (chat), ciascuna identificata da un `chat_id` univoco all'interno dell'intero sistema. Ogni chat contiene una sequenza di messaggi (utente e assistente artificiale), memorizzati in DynamoDB. Per ottimizzare le prestazioni e ridurre il carico di rete, le operazioni di recupero dello storico restituiscono esclusivamente le informazioni di sintesi (ad esempio titolo e metadata), mentre il contenuto completo viene fornito solo su richiesta esplicita.

Quando l'utente invia un messaggio, il sistema non si limita a inoltrarlo direttamente al modello, ma recupera le informazioni più rilevanti dalla chat in corso. In particolare, la Lambda recupera il contesto rilevante della conversazione dalla base dati (DynamoDB), costruendo un prompt arricchito che include la cronologia significativa dei messaggi. Per estrarre tali informazioni, la Lambda utilizza tecniche di ricerca lessicale basate su BM25, evitando di concatenare l'intera conversazione al testo inviato dall'utente. Questo approccio consente di migliorare la coerenza e la pertinenza delle risposte generate dal modello.

Il prompt così costruito viene inviato a Bedrock, che restituisce la risposta generata dal LLM. La risposta, insieme al messaggio dell'utente, viene quindi memorizzata in DynamoDB come parte della chat. Nel caso in cui il messaggio inviato sia il primo di una nuova chat, la Lambda imposta il titolo della chat alle prime 3 parole del messaggio dell'utente. Il flusso di elaborazione è dunque il seguente:

- **Invio della Richiesta:** L'Utente interagisce con il chatbot tramite app, generando una richiesta HTTPS verso uno degli endpoint disponibili (ad esempio `POST /chats/{chat_id}/messages`).
- **Autenticazione:** API Gateway verifica che l'utente sia autenticato all'applicazione. In caso positivo, la richiesta viene inoltrata alla funzione Lambda responsabile della logica del chatbot.
- **Gestione della Chat:** La Lambda identifica la chat di riferimento tramite `chat_id`. Se necessario, recupera i metadati o verifica l'esistenza della chat in DynamoDB.
- **Recupero del Contesto (RAG):** Prima di inviare il messaggio al modello, la Lambda esegue una query su DynamoDB per ottenere i messaggi precedenti rilevanti della conversazione, costruendo il contesto da includere nel prompt.



- **Costruzione del Prompt:** Il messaggio dell'utente viene combinato con il contesto recuperato, formando un prompt strutturato da inviare al modello linguistico.
- **Invocazione del Modello:** La Lambda invia il prompt a Bedrock, che elabora la richiesta e restituisce una risposta generata.
- **Persistenza dei Messaggi:** La Lambda salva sia il messaggio dell'utente sia la risposta del modello in DynamoDB, associandoli alla chat corrente.
- **Aggiornamento dei Metadati:** Se necessario, la Lambda aggiorna le informazioni della chat (ad esempio titolo o timestamp dell'ultima attività).
- **Restituzione della Risposta:** La risposta generata dal modello, insieme ai metadati aggiornati, viene restituita al client.
- **Recupero dello Storico:** Quando l'utente richiede la lista delle chat (metodo `GET /chats/`), la Lambda restituisce solo le *preview* delle chat, senza includere l'intero contenuto dei messaggi.
- **Recupero di una Chat:** Per ottenere il contenuto completo di una chat (metodo `GET /chats/{chat_id}`), la Lambda recupera tutti i messaggi associati da DynamoDB e li restituisce al client.
- **Operazioni CRUD:** Le operazioni di creazione, aggiornamento ed eliminazione delle chat vengono gestite direttamente dalla Lambda tramite interazioni con DynamoDB.

API associate

- `GET /chats/`: ritorna le *preview* di tutte le chat dell'utente.
- `POST /chats/`: crea una nuova chat per l'utente.
- `GET /chats/{chat_id}`: ritorna la chat specificata con il contenuto completo dei messaggi.
- `PUT /chats/{chat_id}`: aggiorna i dati della chat specificata.
- `DELETE /chats/{chat_id}`: rimuove la chat specificata dal database.
- `POST /chats/{chat_id}/messages`: invia un messaggio al modello linguistico; ritorna la risposta generata e il titolo aggiornato della chat.



3.5.6 Contatti fidati, invio SOS e Dead man's switch

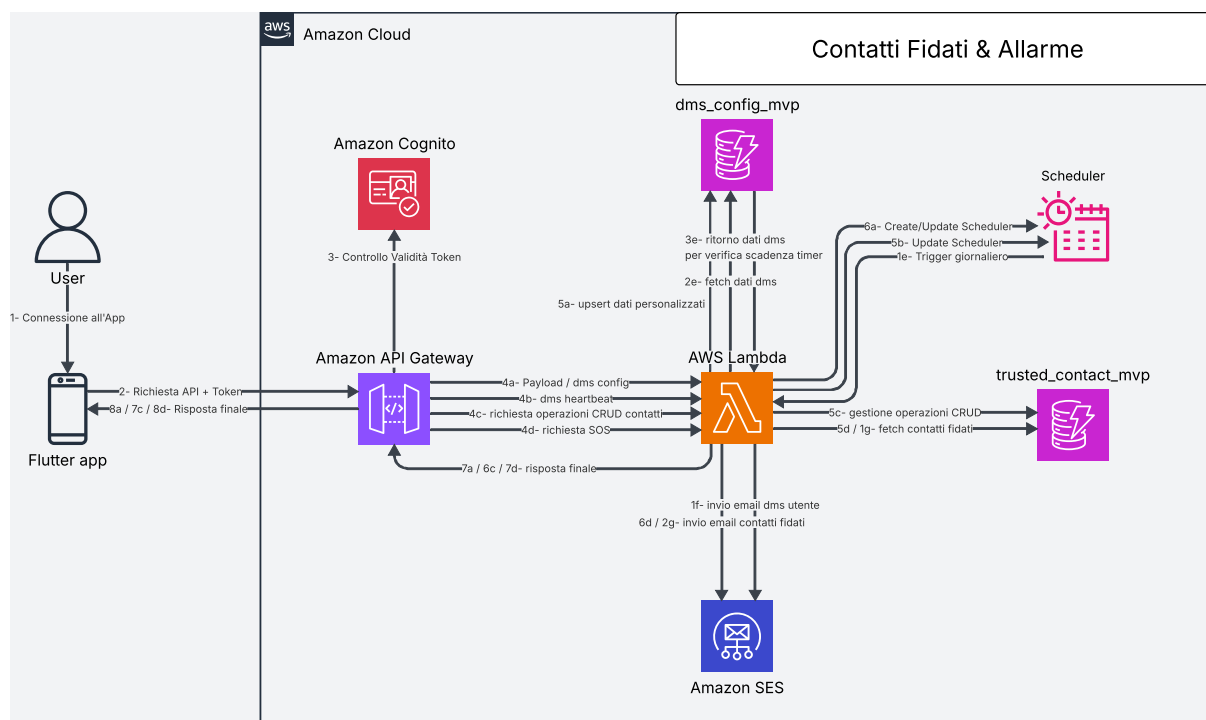


Figura 120: **Architettura^G** del sistema di contatti fidati, SOS e Dead man's switch

Architettura^G e Flusso di Elaborazione: contatti fidati, SOS e Dead man's switch

La sezione relativa ai contatti fidati, all'invio di messaggi di emergenza e al meccanismo di Dead man's switch è progettata per garantire la gestione automatizzata dello stato di attività dell'utente e la possibilità di attivare procedure di emergenza sia manualmente che in modo automatico.

API Gateway gestisce l'esposizione delle API e verifica, per ogni richiesta, che l'utente sia correttamente autenticato all'applicazione. Le richieste valide vengono inoltrate a una funzione Lambda centrale, responsabile della gestione della logica applicativa relativa ai contatti fidati, alla configurazione del Dead man's switch e all'orchestrazione dell'invio delle email tramite Amazon SES.

I dati sono memorizzati su due database DynamoDB distinti:

- **dms_config_mvp**: contiene la configurazione del Dead man's switch, includendo il titolo e il corpo dell'email di inattività, il primo e secondo timer di inattività, l'ultimo accesso dell'utente e lo stato corrente del sistema (attivo, prima inattività, seconda inattività).
- **trusted_contact_mvp**: memorizza le informazioni relative ai contatti fidati associati a ciascun utente, inclusi nome, email e numero di telefono.



Il sistema implementa inoltre un meccanismo di monitoraggio automatico basato su AWS EventBridge. Uno scheduler giornaliero attiva la Lambda centrale che verifica lo stato di attività degli utenti e determina eventuali condizioni di inattività prolungata.

Il flusso di elaborazione del Dead man's switch è il seguente:

- **Trigger programmato:** EventBridge attiva quotidianamente la Lambda, che avvia il controllo dello stato degli utenti.
- **Recupero configurazioni:** la Lambda effettua il fetch dei dati presenti in `dms_config_mvp`, ottenendo i timer di inattività, l'ultimo accesso registrato e lo stato corrente del Dead man's switch.
- **Valutazione dello stato di inattività:** la funzione confronta il tempo corrente con l'ultimo accesso dell'utente. Se viene superata la prima soglia di inattività, il sistema entra nello stato di "prima inattività"; Al raggiungimento della seconda soglia di inattività, viene attivato lo stato di "seconda inattività".
- **Invio email di prima inattività:** al superamento della prima soglia, la Lambda invia tramite Amazon SES un'email all'utente utilizzando il titolo e il corpo configurati, notificando lo stato di inattività.
- **Attivazione SOS (seconda inattività):** al superamento della seconda soglia, la Lambda recupera l'elenco dei contatti fidati da `trusted_contact_mvp` e invia, tramite SES, un'email di SOS predefinita a ciascun contatto.
- **Aggiornamento stato:** lo stato del sistema viene aggiornato in DynamoDB per evitare invii ripetuti non necessari.

Oltre al flusso automatico, il sistema supporta l'invio manuale di SOS e la gestione in tempo reale dello stato di attività dell'utente tramite heartbeat.

Il flusso di heartbeat è il seguente:

- **Invio heartbeat:** All'avvio dell'app, il client invia una richiesta alla API `POST /dms/heartbeat/` per notificare l'utilizzo attivo dell'applicazione.
- **Aggiornamento stato backend:** la Lambda aggiorna il campo "ultimo accesso" nella tabella `dms_config_mvp` e, se necessario, resetta lo stato di inattività.

L'invio SOS **manuale** segue invece il seguente flusso:

- **Richiesta di emergenza:** il client invia una richiesta HTTPS alla API `POST /trusted_contacts/sos/`.
- **Recupero contatti:** la Lambda recupera tutti i contatti fidati associati all'utente da `trusted_contact_mvp`.



- **Invio email SES:** la funzione Lambda invia immediatamente una email di emergenza a tutti i contatti fidati tramite Amazon SES.

La gestione dei contatti fidati è completamente CRUD e permette all'utente di mantenere aggiornato il proprio elenco di destinatari di emergenza.

API associate

- **GET /dms/:** restituisce tutte le opzioni configurabili del Dead man's switch, incluse soglie di inattività e template email.
- **POST /dms/:** salva o aggiorna la configurazione del Dead man's switch dell'utente.
- **POST /dms/heartbeat/:** registra il segnale di attività dell'utente aggiornando l'ultimo accesso.
- **GET /trusted_contacts/:** restituisce la lista dei contatti fidati associati all'utente.
- **POST /trusted_contacts/:** crea un nuovo contatto fidato.
- **PUT /trusted_contacts/{trusted_contact_id}/:** aggiorna un contatto fidato esistente.
- **DELETE /trusted_contacts/{trusted_contact_id}/:** elimina un contatto fidato.
- **POST /trusted_contacts/sos/:** attiva manualmente l'invio di una email di emergenza a tutti i contatti fidati.

4. Stato dei requisiti funzionali

4.1. Requisiti Funzionali obbligatori

Codice	Descrizione	Stato
RF-OB-1	L' Utente^G deve poter autenticarsi all'applicazione tramite l'utilizzo di Google Auth	Non soddisfatto
RF-OB-2	Il sistema deve attivare automaticamente la funzionalità di Dead Man's Switch a seguito del login dell' Utente^G	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-3	L' Utente^G deve poter visualizzare un messaggio di errore esplicativo nel caso in cui l'autenticazione tramite Google Auth fallisca, venga annullata o restituisca parametri non validi	Non soddisfatto
RF-OB-4	L' Utente^G deve poter selezionare il profilo di utilizzo ("Per me" o "Per altri")	Non soddisfatto
RF-OB-5	L' Utente^G deve poter visualizzare la lista dei contatti fidati	Non soddisfatto
RF-OB-6	L' Utente^G deve poter visualizzare un messaggio di errore esplicativo nel caso in cui il recupero dei dati dal database fallisca	Non soddisfatto
RF-OB-7	L' Utente^G deve poter ricaricare la sezione dei contatti fidati dopo la visualizzazione dell'errore di caricamento dei dati dal database	Non soddisfatto
RF-OB-8	Il sistema deve permettere la visualizzazione di un singolo contatto fidato dalla lista dei contatti fidati	Non soddisfatto
RF-OB-9	L' Utente^G deve poter visualizzare la preview delle informazioni (il nome) del contatto fidato che sta visualizzando dalla lista dei contatti fidati	Non soddisfatto
RF-OB-10	L' Utente^G deve poter visualizzare i dettagli di un contatto fidato selezionato dalla lista dei contatti fidati	Non soddisfatto
RF-OB-11	L' Utente^G deve poter visualizzare il nome del contatto fidato selezionato dalla lista dei contatti fidati	Non soddisfatto
RF-OB-12	L' Utente^G deve poter visualizzare il numero di telefono del contatto fidato selezionato dalla lista dei contatti fidati	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-13	L'Utente ^G deve poter visualizzare l'indirizzo email del contatto fidato selezionato dalla lista dei contatti fidati	Non soddisfatto
RF-OB-14	L'Utente ^G deve poter aggiungere un nuovo contatto fidato alla lista dei contatti fidati	Non soddisfatto
RF-OB-15	L'Utente ^G deve poter inserire il nome del nuovo contatto fidato durante l'aggiunta di esso alla lista dei contatti fidati	Non soddisfatto
RF-OB-16	L'Utente ^G deve poter inserire il numero di telefono del nuovo contatto fidato durante l'aggiunta di esso alla lista dei contatti fidati	Non soddisfatto
RF-OB-17	L'Utente ^G deve poter inserire l'indirizzo email del nuovo contatto fidato durante l'aggiunta di esso alla lista dei contatti fidati	Non soddisfatto
RF-OB-18	Il sistema deve riconoscere e gestire le informazioni di numero di telefono e email già presenti nel sistema	Non soddisfatto
RF-OB-19	L'Utente ^G deve poter visualizzare un messaggio di errore esplicativo nel caso in cui le informazioni di numero di telefono e email siano già presenti nel sistema	Non soddisfatto
RF-OB-20	L'Utente ^G deve poter reinserire le informazioni di numero di telefono e email dopo la visualizzazione del messaggio di errore esplicativo	Non soddisfatto
RF-OB-21	Il sistema deve validare la conformità del numero di telefono inserito o modificato	Non soddisfatto
RF-OB-22	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui il numero di telefono inserito o modificato non sia conforme ai criteri prestabiliti	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-23	Il sistema deve validare la conformità dell'indirizzo email inserito o modificato	Non soddisfatto
RF-OB-24	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui l'indirizzo email inserito o modificato non sia conforme ai criteri prestabiliti	Non soddisfatto
RF-OB-25	L' Utente^G deve poter modificare i dati di un contatto fidato esistente	Non soddisfatto
RF-OB-26	L' Utente^G deve poter modificare il nome di un contatto fidato esistente	Non soddisfatto
RF-OB-27	L' Utente^G deve poter modificare il numero di telefono di un contatto fidato esistente	Non soddisfatto
RF-OB-28	L' Utente^G deve poter modificare l'indirizzo email di un contatto fidato esistente	Non soddisfatto
RF-OB-29	L' Utente^G deve poter eliminare un contatto fidato	Non soddisfatto
RF-OB-30	Il sistema deve richiedere conferma prima dell'eliminazione di un contatto fidato	Non soddisfatto
RF-OB-31	L' Utente^G deve poter confermare, con un'apposita voce, l'eliminazione di un contatto fidato quando il sistema richiede la conferma	Non soddisfatto
RF-OB-32	L' Utente^G deve poter effettuare il logout dopo essersi autenticato all'applicazione	Non soddisfatto
RF-OB-34	L' Utente^G deve poter modificare le informazioni della mail di avviso	Non soddisfatto
RF-OB-35	L' Utente^G deve poter modificare il titolo della mail di avviso	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-36	L'Utente ^G deve poter modificare il corpo della mail di avviso	Non soddisfatto
RF-OB-37	L'Utente ^G deve poter visualizzare un'anteprima della mail di avviso	Non soddisfatto
RF-OB-38	L'Utente ^G deve poter visualizzare l'anteprima del corpo della mail di avviso	Non soddisfatto
RF-OB-39	L'Utente ^G deve poter visualizzare un'anteprima del titolo della mail di avviso	Non soddisfatto
RF-OB-40	L'Utente ^G deve poter attivare il Dead Man's Switch	Non soddisfatto
RF-OB-41	L'Utente ^G deve poter disattivare la funzionalità di Dead Man's Switch	Non soddisfatto
RF-OB-42	L'Utente ^G deve poter impostare una password per il diario criptato	Non soddisfatto
RF-OB-43	Il sistema deve validare la conformità della password del diario criptato secondo i criteri pre-stabiliti	Non soddisfatto
RF-OB-44	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui la password del diario criptato non sia conforme ai criteri pre-stabiliti	Non soddisfatto
RF-OB-45	L'Utente ^G deve poter reinserire la password del diario criptato dopo la visualizzazione del messaggio di errore esplicativo	Non soddisfatto
RF-OB-46	L'Utente ^G deve poter impostare una password per il diario fittizio	Non soddisfatto
RF-OB-47	Il sistema deve validare la conformità e l'unicità della password del diario fittizio rispetto a quella del diario criptato	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-48	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui la password del diario fittizio non sia conforme ai criteri prestabiliti	Non soddisfatto
RF-OB-49	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui la password del diario fittizio sia uguale a quella del diario criptato	Non soddisfatto
RF-OB-50	L'Utente ^G deve poter reinserire la password del diario fittizio dopo la visualizzazione del messaggio di errore esplicativo	Non soddisfatto
RF-OB-51	L'Utente ^G deve poter inserire la password per accedere al diario desiderato	Non soddisfatto
RF-OB-52	L'Utente ^G deve poter inserire la password per accedere al diario criptato	Non soddisfatto
RF-OB-53	L'Utente ^G deve poter inserire la password per accedere al diario fittizio	Non soddisfatto
RF-OB-54	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui la password inserita per l'accesso al diario risulti errata	Non soddisfatto
RF-OB-55	L'Utente ^G deve poter reinserire la password di accesso al diario dopo la visualizzazione del messaggio di errore esplicativo	Non soddisfatto
RF-OB-56	Il sistema deve bloccare temporaneamente l'accesso al diario in seguito al superamento del limite massimo di tentativi errati di inserimento password	Non soddisfatto
RF-OB-57	L'Utente ^G deve poter visualizzare il contenuto del diario criptato	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-58	L'Utente ^G deve poter visualizzare la lista delle note presenti nel diario criptato	Non soddisfatto
RF-OB-59	L'Utente ^G deve poter visualizzare l'anteprima della singola nota all'interno della lista delle note del diario criptato	Non soddisfatto
RF-OB-60	L'Utente ^G deve poter ricaricare la sezione delle note del diario criptato dopo la visualizzazione dell'errore di caricamento dei dati dal database	Non soddisfatto
RF-OB-61	L'Utente ^G deve poter creare una nuova nota vuota all'interno del diario criptato	Non soddisfatto
RF-OB-62	L'Utente ^G deve poter eliminare una nota esistente dal diario criptato	Non soddisfatto
RF-OB-63	Il sistema deve richiedere conferma prima dell'eliminazione definitiva di una nota del diario criptato	Non soddisfatto
RF-OB-64	L'Utente ^G deve poter confermare, con un'apposita voce, l'eliminazione della nota dal diario criptato quando il sistema ne richiede la conferma	Non soddisfatto
RF-OB-65	L'Utente ^G deve poter visualizzare il contenuto completo di una nota selezionata dalla lista delle note del diario criptato	Non soddisfatto
RF-OB-66	L'Utente ^G deve poter visualizzare il contenuto testuale della nota del diario criptato	Non soddisfatto
RF-OB-67	L'Utente ^G deve poter visualizzare il contenuto multimediale della nota del diario criptato	Non soddisfatto
RF-OB-68	L'Utente ^G deve poter visualizzare il contenuto audio presente all'interno della nota del diario criptato	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-69	L'Utente ^G deve poter visualizzare le immagini presenti all'interno della nota del diario criptato	Non soddisfatto
RF-OB-70	L'Utente ^G deve poter inserire del contenuto testuale all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-71	L'Utente ^G deve poter inserire del contenuto multimediale all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-72	L'Utente ^G deve poter inserire un'immagine all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-73	L'Utente ^G deve poter inserire una traccia audio all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-74	L'Utente ^G deve poter modificare il contenuto testuale presente all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-75	L'Utente ^G deve poter rimuovere del contenuto multimediale presente all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-76	L'Utente ^G deve poter rimuovere un'immagine presente all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-77	L'Utente ^G deve poter rimuovere una traccia audio presente all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-78	L'Utente ^G deve poter visualizzare la schermata del diario fittizio	Non soddisfatto
RF-OB-79	L'Utente ^G deve poter visualizzare la lista delle note presenti nel diario fittizio	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-80	L'Utente ^G deve poter visualizzare l'anteprima della singola nota all'interno della lista delle note del diario fittizio	Non soddisfatto
RF-OB-81	L'Utente ^G deve poter ricaricare la sezione delle note del diario fittizio dopo la visualizzazione dell'errore di caricamento dei dati dal database	Non soddisfatto
RF-OB-82	L'Utente ^G deve poter eliminare una nota esistente dal diario fittizio	Non soddisfatto
RF-OB-83	Il sistema deve richiedere conferma prima dell'eliminazione definitiva di una nota del diario fittizio	Non soddisfatto
RF-OB-84	L'Utente ^G deve poter confermare, con un'apposita voce, l'eliminazione della nota dal diario fittizio quando il sistema ne richiede la conferma	Non soddisfatto
RF-OB-85	L'Utente ^G deve poter creare una nuova nota vuota all'interno del diario fittizio	Non soddisfatto
RF-OB-86	L'Utente ^G deve poter visualizzare il contenuto completo di una nota selezionata dalla lista delle note del diario fittizio	Non soddisfatto
RF-OB-87	L'Utente ^G deve poter visualizzare il contenuto testuale della nota del diario fittizio	Non soddisfatto
RF-OB-88	L'Utente ^G deve poter visualizzare il contenuto multimediale della nota del diario fittizio	Non soddisfatto
RF-OB-89	L'Utente ^G deve poter visualizzare il contenuto audio presente all'interno della nota del diario fittizio	Non soddisfatto
RF-OB-90	L'Utente ^G deve poter visualizzare le immagini presenti all'interno della nota del diario fittizio	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-91	L' Utente^G deve poter inserire del contenuto testuale all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-92	L' Utente^G deve poter inserire del contenuto multimediale all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-93	L' Utente^G deve poter inserire un'immagine all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-94	L' Utente^G deve poter inserire una traccia audio all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-95	L' Utente^G deve poter rimuovere del contenuto multimediale presente all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-96	L' Utente^G deve poter rimuovere un'immagine presente all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-97	L' Utente^G deve poter rimuovere una traccia audio presente all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-98	L' Utente^G deve poter modificare il contenuto testuale presente all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-99	L' Utente^G deve poter inviare un messaggio di SOS ai contatti fidati tramite un'apposita voce	Non soddisfatto
RF-OB-100	Il sistema deve notificare l' Utente^G dell'assenza di connessione internet durante il tentativo di invio del messaggio SOS e proporre l'invio di un segnale acustico	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-101	Il sistema deve notificare l' Utente^G della mancanza di contatti fidati configurati durante il tentativo di invio del messaggio SOS e proporre l'invio di un segnale acustico	Non soddisfatto
RF-OB-102	L' Utente^G deve poter avviare un segnale acustico di emergenza indipendente dalle impostazioni di volume del dispositivo	Non soddisfatto
RF-OB-103	L' Utente^G deve poter inviare un messaggio al chatbot e ricevere una risposta generata dall'LLM in base al contesto scelto	Non soddisfatto
RF-OB-104	L' Utente^G deve poter interrompere la generazione della risposta dell'LLM durante la consultazione del chatbot	Non soddisfatto
RF-OB-105	Il sistema deve notificare l' Utente^G della mancanza di connessione internet durante la consultazione del chatbot	Non soddisfatto
RF-OB-106	L' Utente^G deve poter ripetere l'invio del messaggio o scriverne uno nuovo dopo la notifica di assenza di connessione internet	Non soddisfatto
RF-OB-107	L' Utente^G deve poter cambiare il contesto della chat a "Detective delle Relazioni"	Non soddisfatto
RF-OB-108	L' Utente^G deve poter cambiare il contesto della chat a "Specchio Intelligente"	Non soddisfatto
RF-OB-109	Il sistema deve notificare l' Utente^G di un errore durante il cambio di contesto del chatbot e mantenere il contesto precedente	Non soddisfatto
RF-OB-110	L' Utente^G deve poter ritentare il cambio di contesto del chatbot dopo la notifica di errore	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-111	L' Utente^G deve poter creare una nuova sessione di chat con il chatbot	Non soddisfatto
RF-OB-112	L' Utente^G deve poter eliminare una chat dalla cronologia delle chat	Non soddisfatto
RF-OB-113	Il sistema deve richiedere conferma esplicita prima dell'eliminazione definitiva di una chat	Non soddisfatto
RF-OB-114	L' Utente^G deve poter confermare, con un'apposita voce, l'eliminazione di una chat quando il sistema ne richiede la conferma	Non soddisfatto
RF-OB-115	L' Utente^G deve poter visualizzare il contenuto di una chat selezionata, inclusa la lista dei messaggi scambiati	Non soddisfatto
RF-OB-116	Il sistema deve notificare l' Utente^G di un errore nel caricamento del contenuto della chat	Non soddisfatto
RF-OB-117	Il sistema deve visualizzare la lista dei messaggi scambiati tra l' Utente^G e il chatbot all'interno di una chat	Non soddisfatto
RF-OB-118	Il sistema deve distinguere visivamente i messaggi inviati dall' Utente^G da quelli generati dal chatbot	Non soddisfatto
RF-OB-119	Il sistema deve visualizzare i messaggi inviati dall' Utente^G con la formattazione grafica allineata a destra	Non soddisfatto
RF-OB-120	Il sistema deve visualizzare i messaggi generati dal chatbot con la formattazione grafica allineata a sinistra	Non soddisfatto
RF-OB-121	L' Utente^G deve poter visualizzare la cronologia delle chat con il chatbot	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-122	L'Utente ^G deve poter selezionare e visualizzare una singola chat dalla cronologia	Non soddisfatto
RF-OB-123	Il sistema deve mostrare il titolo identificativo di ogni chat presente nella cronologia	Non soddisfatto
RF-OB-124	L'Utente ^G deve poter visualizzare la lista delle community disponibili	Non soddisfatto
RF-OB-125	Il sistema deve notificare l'Utente ^G di un errore nel caricamento della lista delle community	Non soddisfatto
RF-OB-126	Il sistema deve mostrare i dati riassuntivi di ogni singola community nella lista	Non soddisfatto
RF-OB-127	Il sistema deve mostrare il nome di ogni singola community nella lista	Non soddisfatto
RF-OB-128	Il sistema deve mostrare il link di ogni singola community nella lista	Non soddisfatto
RF-OB-129	L'Utente ^G deve poter aprire il link di una community per visitarne il sito web esterno	Non soddisfatto
RF-OB-130	L'Utente ^G deve poter visualizzare la lista delle leggi e normative disponibili	Non soddisfatto
RF-OB-131	Il sistema deve mostrare il titolo di ogni singola legge nella lista	Non soddisfatto
RF-OB-132	L'Utente ^G deve poter visualizzare i dettagli completi di una legge selezionata dalla lista	Non soddisfatto
RF-OB-133	Il sistema deve notificare l'Utente ^G di un errore nel caricamento dei dati di una legge	Non soddisfatto
RF-OB-134	L'Utente ^G deve poter visualizzare la lista dei contatti e degli enti d'emergenza	Non soddisfatto
RF-OB-135	Il sistema deve mostrare i dati riassuntivi di ogni singolo contatto d'emergenza nella lista	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-136	Il sistema deve mostrare il nome identificativo di ogni contatto d'emergenza	Non soddisfatto
RF-OB-137	Il sistema deve mostrare il numero di telefono associato ad ogni contatto d'emergenza	Non soddisfatto
RF-OB-139	L' Utente^G deve poter visualizzare la lista del materiale informativo disponibile	Non soddisfatto
RF-OB-140	Il sistema deve notificare l' Utente^G di un errore nel caricamento della lista del materiale informativo	Non soddisfatto
RF-OB-141	Il sistema deve mostrare il titolo e il formato di ogni singolo materiale informativo nella lista	Non soddisfatto
RF-OB-142	L' Utente^G deve poter visualizzare il contenuto di un materiale informativo selezionato dalla lista	Non soddisfatto
RF-OB-143	Il sistema deve notificare l' Utente^G di un errore nel recupero dei dati del materiale informativo	Non soddisfatto
RF-OB-144	L' Utente^G deve poter visualizzare la mappa interattiva con i marker dei luoghi sicuri nelle vicinanze	Non soddisfatto
RF-OB-145	L' Utente^G deve poter aprire un luogo sicuro nella mappa esterna Google Maps	Non soddisfatto
RF-OB-146	Il sistema deve notificare l' Utente^G di un errore nell'apertura dell'applicazione Google Maps e mantenere la visualizzazione della mappa interna	Non soddisfatto

Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori



4.2. Requisiti Funzionali opzionali

Codice	Descrizione	Riferimento
RF-OP-1	L' Utente^G deve poter impostare il timer di prima inattività (per la funzionalità di Dead Man's Switch)	Non Soddisfatto
RF-OP-2	L' Utente^G deve poter impostare il timer di seconda inattività (per la funzionalità di Dead Man's Switch)	Non Soddisfatto
RF-OP-3	Il sistema deve inviare un SMS ai numeri di telefono dei contatti fidati all'invio del segnale SOS	Non Soddisfatto
RF-OP-4	Il sistema deve indicare all' Utente^G se le risorse sono autorevoli e verificate	Non Soddisfatto
RF-OP-5	Il sistema deve mettere a disposizione leggi estere	Non Soddisfatto
RF-OP-6	Il sistema deve mostrare leggi aggiornate, giuridicamente valide e complete	Non Soddisfatto
RF-OP-7	Il sistema deve indicare all' Utente^G se i luoghi sicuri visualizzati nella mappa sono autorevoli e verificati	Non Soddisfatto

Tabella 3: Tabella dello stato dei requisiti funzionali opzionali